

Do Large Language Models Exploit Decisive Sequential Information?

Anonymous Full Paper
Submission 42

Abstract

Can Large Language Models (LLMs) capture sequential structure when outcomes depend causally on order? We study this within a controlled, semantic-free sequence framework that instantiates two mechanisms: ordered-compliance (driven by strict sequential order) and global-property-compliance (driven by a global property). Theoretically, we prove that ordered-compliance inherently leaks information through frequency patterns, creating predictive shortcuts that allow models to succeed without truly capturing causal sequential structure. To isolate true order learning, we introduce a shortcut-auditing protocol that quantifies the predictive power of non-sequential features (e.g., k -grams, pair counts) under label noise. Empirically, encoder and recurrent models reach the theoretical optimum (measured by AUC and F1) on ordered-compliance more efficiently than fine-tuned decoder-only LLMs. Conversely, on global-property-compliance, all architectures fail under standard end-to-end training unless explicitly guided on feature relevance. Our results demonstrate that high performance on order-sensitive tasks often masks shortcut exploitation, highlighting the critical need for rigorous shortcut auditing.

1 Introduction

Consider a clinician reviewing a patient’s medical history to predict disease progression. The sequence of symptoms matters: *diabetes mellitus* followed by *visual disturbance* tells a different story than the same conditions appearing in reverse order for predicting *myocardial infarction* [1]. Similarly, in recommendation systems, process tracing, and transaction streams, labels may depend not only on *which* events occurred, but also on the *order* in which they occurred. Recent electronic health record (EHR) studies use comparator cohorts that preserve event content while altering temporal structure, and show that sequence-based disease representations consistently outperform bag-of-events models by leveraging the prognostic value of directional disease progression [1–4]. In recommendation, the same preferred items can reflect different user preferences when encountered in different orders [5].

Suppose a sequential model outperforms a bag-of-events model on clinical trajectories. Has it really

learned to use order? Not necessarily. Real sequential data contain many overlapping signals: which events occur, how often, how recently, and which ones tend to appear together. A model can use any of these, or fall back on local pattern matching and lagged pair statistics, without ever recovering the causal latent trajectory rule. The difficulty is that real-world event trajectories rarely come with a known underlying mechanism, and the space of possible shortcuts is open-ended. The reason to study a controlled setting is therefore not that real-world trajectories are simple, but that they are too entangled to support mechanism attribution directly. We instantiate two complementary mechanisms, both defined over a hidden subset of letters: in *ordered-compliance* (OC), these letters must appear in a secret order at a fixed lag; in *global-property-compliance* (GPC), the label depends on a global property of their counts.

As LLMs and transformer-based sequential models are increasingly applied to clinical event prediction, sequential recommendation, and temporal reasoning [6–10], the relevant question is not only whether they can use order, but also whether their performance supports a claim about the mechanism they learned. Recent evidence motivates this concern: LLM-based recommenders often retain most of their performance even when item order is shuffled [5]; clinical analyses raise concerns about statistical pattern matching rather than faithful temporal reasoning [11]; and transformer predictions can often be approximated by simple k -gram statistics [12].

We study this problem with a controlled letter-sequence framework in which semantic meaning and world knowledge are deliberately removed. The abstraction preserves two structural ingredients common to real-world event trajectories: only a hidden subset of letters is relevant, and labels depend on structure among those letters. A consequence of this setup, made precise in Section 3, is that simpler local or lagged statistics may already carry predictive signal even when the underlying mechanism is sequential; exactly the entanglement that makes shortcut auditing necessary. We prove that for non-degenerate ordered-compliance rules sampled i.i.d. with full support, individual positions inevitably carry information about the label, formalized through *single-position local purity*. The empirical question therefore shifts: rather than ask-

ing whether models use sequential structure at all, we ask how much of the label is already recoverable from local and lagged statistics, and which architectures recover them efficiently from raw sequences.

To answer this, we compare decoder-only LLMs, encoder-style transformers, and recurrent architectures against a graded series of baselines that includes contiguous k -grams and λ -aware pair-count classifiers. Three findings emerge: encoder-style and recurrent models reach the theoretical optimum on OC with substantially fewer parameters and less data than decoder-only LLMs; under label noise this gap widens, with decoder-only models dropping to the k -gram level up to 8B parameters; and on GPC all architectures remain at chance under end-to-end training, but a 64-unit MLP solves the task once subset membership is provided. Finally, a real-world clinical audit validates our core premise: even temporal dependencies can be heavily shortcut-dominated.

Contributions.

- **Theory.** For OC and GPC, two label-generating mechanisms over letter sequences, we prove that they have opposite local-information structure: OC inevitably leaks single-position information, whereas balanced GPC is invisible to every proper subset of positions.
- **Shortcut-auditing protocol.** We quantify how much outcome signal is recoverable from unordered counts, contiguous k -grams, lag-aware pair counts, and pair counts over all possible lags, with closed-form expressions for the theoretical optimum under label noise.
- **Mechanism attribution.** Across decoder-only LLMs, encoder-style transformers, and recurrent models, the protocol identifies what each architecture actually learns: on OC, performance is explained by lag-aware pair-count features; on GPC, the protocol isolates subset identification as the limiting sub-task.
- **Real-world audit.** We apply the protocol to a real clinical prediction task, demonstrating that apparent temporal sequences in real-world data can be heavily shortcut-dominated.

2 Related work

A growing body of work in domains such as healthcare and recommendation suggests that temporal order can add predictive value beyond unordered event identity [2–5], and transformer-based models are now routinely applied to longitudinal event histories [6–8, 10]. Of particular relevance, Edelman et al. [1] study interpretable $A \rightarrow B \rightarrow C$ clinical trajectories using comparator cohorts that preserve

event content while changing temporal structure, including B without prior A , reverse-order, and unordered comparisons. Such settings, together with event-log domains such as predictive process monitoring and system-log anomaly detection [13, 14], motivate our question but cannot answer it directly: sequence-model gains may reflect temporal order, event semantics, co-occurrence, recency, or local event-pattern shortcuts rather than global trajectory reasoning.

Several recent benchmarks probe temporal or sequential ability [9, 15], but Klenitskiy et al. [16] argue that many datasets marketed as sequential do not in fact require strong sequential modeling. Consistent with this, transformer behavior is often well approximated by local statistics or k -gram rules [5, 11, 12], connecting to the broader shortcut-learning literature [17–19]. At the same time, sequential models can in principle realize dependencies beyond fixed k -gram rules [20, 21].

Recent work has introduced tunable synthetic benchmarks for reasoning and sequence processing [22, 23]. Our framework differs by stripping away semantics while enabling a theoretical analysis of the local-information structure of each mechanism: OC suffers unavoidable local leakage, whereas balanced GPC is invisible to every proper subset of positions. We pair the framework with a shortcut-auditing protocol that quantifies which feature families explain each model’s predictions.

3 Methods and theory

In this section, we formalize the notion of *pure sequential dependence*, introduce the two experimental mechanisms of our framework, and prove theoretical properties characterizing each mechanism with respect to this property.

Illustrative example. We abstract temporal signals from domains like EHRs or recommendation into semantic-free sequences. Here, only a hidden subset of letters is predictive, and the label depends on their relative order under a rule κ at a fixed lag λ . Our shortcut-auditing protocol tests whether models recover these latent parameters or merely exploit simpler correlates. A clinical analogy and concrete sequence walk-through are deferred to Appendix A.

3.1 Setup and notation

Let $\mathcal{A}$ be an alphabet of size $\ell = |\mathcal{A}|$. Consider a hidden key subset $\mathcal{S} \subseteq \mathcal{A}$ of size $m = |\mathcal{S}|$ ($m \geq 1$) uniformly among all m -subsets of $\mathcal{A}$, and then sample a bijection $\kappa : \mathcal{S} \rightarrow \{1, \dots, m\}$ uniformly among all such bijections. The realized pair $(\mathcal{S}, \kappa)$ is fixed for that particular mechanism; all probabilities below

are conditional on this realized pair unless stated otherwise. Symbols in $\mathcal{A} \setminus \mathcal{S}$ act as distractors and play no role in determining compliance. Let P_0 denote the proposal distribution over candidate sequences of length n : $X_t \stackrel{\text{iid}}{\sim} \text{Uniform}(\mathcal{A})$, $t = 1, \dots, n$. This i.i.d. uniform law is the base synthetic generation mechanism.

Definition 3.1 (Ordered-Compliance Mechanism). Fix a lag parameter $\lambda \geq 1$. A λ -run of key symbols in a sequence $X = (X_1, \dots, X_n)$ is a tuple of positions

$$r = (t, t + \lambda, \dots, t + (q - 1)\lambda), \quad q \geq 1,$$

such that $t \geq 1$ and $t + (q - 1)\lambda \leq n$, and

$$X_{t+i\lambda} \in \mathcal{S} \quad \text{for every } i = 0, \dots, q - 1.$$

The run is *maximal* if it cannot be extended to the left or to the right by another key symbol at distance λ ; that is,

$$\begin{aligned} t - \lambda < 1 \quad \text{or} \quad X_{t-\lambda} \notin \mathcal{S}, \quad \text{and} \\ t + q\lambda > n \quad \text{or} \quad X_{t+q\lambda} \notin \mathcal{S}. \end{aligned}$$

A maximal λ -run $r = (t, t + \lambda, \dots, t + (q - 1)\lambda)$ is *ordered* if $q \geq 2$ and

$$\kappa(X_t) \leq \kappa(X_{t+\lambda}) \leq \dots \leq \kappa(X_{t+(q-1)\lambda}).$$

We say that X is *ordered-compliant* (OC) if it contains at least one ordered maximal λ -run.

Let $Y^* = \mathbf{1}\{X \text{ is compliant}\}$, $\pi \in [0, 1/2)$, and Y denote the observed label, generated from Y^* according to

$$P(Y = 1 \mid Y^* = y^*) = \begin{cases} 1 - \pi, & y^* = 1, \\ \pi, & y^* = 0. \end{cases} \quad (1)$$

For the experiments in Sections 4 and 5, we generate an initial oversized pool of sequences i.i.d. from P_0 and compute the deterministic latent labels Y^* . We then stratify on Y^* to create train/validation/test splits and subsamples of varying size, strictly preserving the latent prevalence reported in Table 1. Next, we assign the final labels Y according to (1): for the deterministic setting ($\pi = 0$), $Y = Y^*$ whereas for the noisy setting ($\pi \in (0, 1/2)$), we independently flip the labels after stratification. Note that post-stratification, the empirical distribution of the retained sequences need not be an i.i.d. sample from P_0 with exactly uniform marginals. Thus, our theoretical results apply to the proposal law P_0 , requiring only independence and full support.

3.2 Pure sequential dependence

We use the term *pure sequential dependence* informally to denote order-sensitive tasks without local shortcuts. Since this phrase mixes order-sensitivity with distributional independence, we formalize the latter through J -local purity.

Definition 3.2 (Local purity). Let $X = (X_1, \dots, X_n) \in \mathcal{A}^n$ and $Y \in \{0, 1\}$ be distributed according to P . For $J \subseteq [n] = \{1, \dots, n\}$, write $X_J = (X_j)_{j \in J}$, which takes values in $\mathcal{A}^J$. A task is J -locally pure under P if

$$I_P(Y; X_J) = 0.$$

It is k -locally pure if

$$I_P(Y; X_J) = 0 \quad \text{for every } J \subseteq [n] \text{ with } 1 \leq |J| \leq k$$

In particular, 1-local purity is equivalent to

$$I_P(Y; X_j) = 0 \quad \text{for every } j \in [n].$$

For finite alphabets,

$$I_P(Y; X_J) = \sum_{y \in \{0,1\}} \sum_{x_J \in \mathcal{A}^J} P(Y = y, X_J = x_J) \log \frac{P(Y = y, X_J = x_J)}{P(Y = y)P(X_J = x_J)}.$$

Theorem 3.3 shows that the ordered-compliance mechanism of Definition 3.1 is not even 1-locally pure: there exists a single coordinate, X_1 under the theorem's indexing, such that $I_P(Y; X_1) > 0$.

Theorem 3.3 (OC Mechanism Induces Local Information). Fix a realized pair $(\mathcal{S}, \kappa)$ as in Section 3.1. Let $\lambda \geq 1$ and $n \geq \lambda + 1$. Let $X = (X_1, \dots, X_n)$ have independent coordinates with full support on $\mathcal{A}$, i.e. $P(X_t = a) > 0$ for all t and all $a \in \mathcal{A}$. Define $Y^* = \mathbf{1}\{X \text{ is OC}\}$ and let Y be the observed label generated from Y^* according to (1). Let $\rho := P(Y^* = 1)$. If $\rho \in (0, 1)$, then there exist $a, b \in \mathcal{A}$ such that $P(Y^* = 1 \mid X_1 = a) \neq P(Y^* = 1 \mid X_1 = b)$. Consequently, $I(Y^*; X_1) > 0$. Moreover, $P(Y = 1 \mid X_1 = a) \neq P(Y = 1 \mid X_1 = b)$ and therefore $I(Y; X_1) > 0$.

The proof couples two sequences differing only at position 1 and shows the resulting posteriors differ; the symmetric-noise channel maps $u = P(Y^* = 1 \mid X_1 = a)$ to $\pi + (1 - 2\pi)u$, strictly monotone for $\pi < 1/2$, so the dependence carries over to Y . See Appendix E.1. Hence, non-degenerate OC mechanisms under independent full-support sampling are not 1-locally pure.

3.3 A complementary global-property-compliance mechanism

While ordered-compliance inevitably leaks single-position information (Theorem 3.3), we now introduce a complementary mechanism with the opposite information structure: global-property-compliance (GPC), in which the label requires information from the full sequence to be recovered.

Definition 3.4 (Global-Property-Compliance). Let $\mathcal{A}$ be an alphabet of size $\ell = |\mathcal{A}|$. Fix a non-empty proper subset $K \subsetneq \mathcal{A}$. Let $X = (X_1, \dots, X_n)$ with $X_t \stackrel{\text{iid}}{\sim} \text{Uniform}(\mathcal{A})$. For $k \in K$, define the count $N_k := \sum_{t=1}^n \mathbf{1}\{X_t = k\}$. The target label is $Y^* := \mathbf{1}\{| \{k \in K : N_k \equiv 0 \pmod{2} \} | \equiv 0 \pmod{2}\}$.

Hence, $Y^* = 1$ iff an even number of key letters have an even occurrence count. Equivalently, Y^* is a fixed bit flip of the parity of $\sum_t \mathbf{1}\{X_t \in K\}$; the challenge is therefore to identify the hidden membership predicate and compute global parity over the resulting membership bits.

The quantity Y^* depends on the full sequence through a global parity computation over a hidden subset. Unlike the compliance family, this construction is not defined through ordered lagged chains.

Theorem 3.5 (*k*-local Purity of the GPC Mechanism). Assume the setting of Definition 3.4 with $p := |K|/|\mathcal{A}|$. For $J \subseteq [n] = \{1, \dots, n\}$, write $X_J = (X_j)_{j \in J}$, which takes values in $\mathcal{A}^J$. Fix $n \geq 1$ and let $J \subseteq [n]$ with $1 \leq |J| < n$. Then $I(Y^*; X_J) = 0$ if $p = 1/2$, and $I(Y^*; X_J) > 0$ if $p \neq 1/2$. Moreover, if $p \neq 1/2$ and $J \subset \mathbb{N}$ is any fixed finite index set, interpreted as a subset of $\{1, \dots, n\}$ for all sufficiently large n , then $I(Y^*; X_J) \rightarrow 0$ as $n \rightarrow \infty$. If the observed label Y is generated from Y^* via (1) with $\pi < 1/2$, the same qualitative statements hold with Y in place of Y^* . In particular, when $p = 1/2$, the parity task is *k*-locally pure for every $k < n$.

See Appendix E.4 for the detailed proof.

4 Experiments

We evaluate whether different architectures recover the hidden mechanism signal beyond contiguous *k*-gram baselines, and then audit whether that signal is explained by lag-aware local features. We then apply the same shortcut-auditing logic to a real diagnosis-code prediction task in MIMIC-IV [24], a widely adopted public database of electronic health records, to test whether a clinically temporal task is also order-informational under a diagnosis-sequence representation. Code is available in an anonymous repository.¹

4.1 Controlled synthetic dataset and task variants

We instantiate the mechanisms of Section 3 on semantic-free sequences of 20 letters over a 26-symbol alphabet. The dataset variants, summarized in Table 1, are designed to separate possible

Table 1. Dataset characteristics for the sequential classification task. N_{train} indicates training size.

Dataset	N_{train}	n	ρ	λ	π	m
OC-Deterministic	400K	20	0.293	7	0	6
OC-Noisy	400K	20	0.293	7	0.3	6
GPC-Deterministic	400K	20	0.499	–	0.0	6

explanations of model success: content-frequency shortcuts, contiguous local patterns, lag-aware pair-count shortcuts, and recovery of the intended hidden mechanism.

We use $(n, \ell, m, \lambda) = (20, 26, 6, 7)$ as a medium-difficulty anchor regime rather than a tuned optimum. With $m = 6$, a sequence contains $nm/\ell \approx 4.6$ key symbols in expectation, so the hidden subset is sparse but not vanishingly rare. With $\lambda = 7$, the ordered relation is non-adjacent while still providing $n - \lambda = 13$ candidate lagged pairs and residue-class runs of length two or three. The resulting latent compliance rate is $\rho \approx 0.293$, giving a non-degenerate classification problem. Appendix H sweeps m and λ and shows that the qualitative conclusions are stable around this anchor.

We instantiate the **OC-Deterministic** ($\pi = 0$) and **OC-Noisy** ($\pi = 0.3$) variants to implement the OC mechanism (Definition 3.1), extending the order-sensitive trajectory paradigm [1, 2] to a controlled synthetic setting. These variants test whether models can discover the hidden subset and lagged ordering rule, and whether their performance exceeds contiguous *k*-gram shortcuts. We also introduce the **GPC-Deterministic** variant to implement the GPC mechanism (Definition 3.4): its label depends on a global modular property over a hidden subset, not on temporal order, a paradigm long used as a stress test for sequence-model expressivity [25]. We use $|K| = 6$ to match the hidden-subset size of the OC variants; the balanced $|K| = 13$ regime is studied in Appendix I.6.

4.2 Models and baselines

We compare three categories of models; Table F.1 in Appendix F summarizes all configurations.

Pretrained LLMs (fine-tuned). We fine-tune Llama-3.2-1B, Llama-3.1-8B and Llama-3.1-70B (the latter only on GPC-Deterministic) [26, 27], Qwen3-4B-Think [28], Qwen2.5-14B [29, 30], and BERT-base (110M) [31] and RoBERTa-large (355M) [32] for binary classification.

Randomly initialized architectures. To isolate architecture from pretrained knowledge, we train Llama-style decoders (1M), LSTM [33], Transformer encoder [34], and a hybrid BiLSTM-Transformer [35]

¹https://anonymous.4open.science/r/anonymous_neurips_seqllm/

(RNNTransformer henceforth), each with 100K–400K parameters.

Traditional baselines. XGBoost [36] with two encodings: ordinal integer-mapped characters (XGBoost-basic) and, following Do et al. [37] and Alshraideh et al. [38], mean-pooled embeddings from Llama-3.2-1B (XGBoost-LLM). Sequences are encoded as a **Sequential events:** $x_1 \dots x_n$ prompt for LLMs/BERT and as learned embeddings for randomly-initialized models (Appendix F.2).

k -gram and lag-aware baselines. Three local-pattern references: (i) a contiguous k -gram classifier that estimates $\hat{P}(Y = 1 | g) = \#(g, Y = 1) / \#(g)$ and averages over the test sequence’s k -grams (Appendix D); (ii) a λ -aware pair-count classifier, logistic regression with L_2 regularization on the 26×26 -dimensional feature $g_{ab}^{(\lambda)}$ with λ given, which controls for whether the task can be solved by a single linear readout once the hidden offset is known; and (iii) a λ -agnostic baseline that stacks $g_{ab}^{(\lambda')}$ across all candidate offsets $\lambda' \in \{1, \dots, n-1\}$ (dimension $(n-1) \times 676 = 12844$) and lets an L_1 -regularized logistic regression or a boosted tree rediscover λ from data. Baseline (iii) is the fair “is the encoder’s advantage about discovering λ ?” test.

4.3 Training, evaluation, and theoretical ceiling

For the controlled synthetic tasks, all neural models are trained with AdamW [39] and cross-entropy loss under early stopping. Pretrained LLMs and BERT are fine-tuned with LoRA [40], using gradient checkpointing and mixed precision for the larger models; randomly initialized models use architecture-specific learning rates and dropout; XGBoost uses regularized boosting with class weighting. We tune the decision threshold on validation to maximize F1. Experiments are run on a single NVIDIA H100 with three seeds; full hyperparameters are in Appendix F. Denoting by N the total amount of data, we write N_{train} , N_{val} , N_{test} for the Training/Validation/Test split, using a ratio of 80%–10%–10%. We report AUC, F1, Precision and Recall on held-out test sets, sweeping training size from 4K to 400K.

For the synthetic mechanisms, we denote by AUC^* the AUC obtained by an oracle score that perfectly recovers the latent noise-free decision rule Y^* ; see Theorem E.1. In deterministic settings ($\pi = 0$), this gives $AUC^* = 1$. For OC-Noisy, using the latent compliance rate $\rho = P(Y^* = 1) = 0.293$ and noise rate $\pi = 0.3$, Theorem E.1 gives $AUC^* = 0.670$. The same theorem also reports the precision, recall, and F1 of the hard oracle classifier $\hat{Y} = Y^*$ evaluated against the stochastic labels: for OC-Noisy these are precision 0.700, recall 0.491, and F1 0.577. These

F1-related quantities are not upper bounds under validation-threshold tuning, so we use AUC as the primary metric. We interpret $AUC \approx 0.50$ as no recovery of the latent rule, $0.55 < AUC < AUC^*$ as partial recovery, and $AUC \approx AUC^*$ as full recovery up to label noise.

4.4 Real-world audit protocol: clinical diagnosis sequences

Moving beyond synthetic settings, motivated by Edelman et al. [1], we sought a real clinical task where temporal structure is plausible but model-mechanism attribution remains directly auditable. We select MIMIC-IV [24, 41] as our clinical testbed, given its status as a foundational machine learning benchmark and its growing adoption for evaluating LLMs in healthcare. We focus on progression from chronic kidney disease (CKD) to end-stage renal disease (ESRD), a staged renal-progression endpoint with inherently temporal clinical structure [42, 43]. CKD defines cohort entry, ESRD defines the future endpoint, and the model observes the pre-endpoint diagnosis history. Our question is not whether CKD progression is clinically temporal, but whether temporal order within that diagnosis history is decisive for model performance under a diagnosis-code representation. We construct a cohort of 14,813 patients (1,146 positives), mapping their specific diagnosis codes to 293 broader Clinical Classifications Software (CCS) categories [44]. To prevent label leakage, we exclude patients who receive both diagnoses during the same admission. Sequences are ordered temporally across admissions and by billing order within them. We audit this task using three shortcut controls: (i) contiguous k -gram diagnostics, (ii) order-invariant bag-of-codes baselines (Logistic Regression, XGBoost), and (iii) sequence models evaluated on randomly shuffled patient sequences. We use AUC as the primary metric, with full cohort definitions provided in Appendix G.1.

5 Results and discussion

We evaluate all models on the three datasets of Table 1; Figure 1 and Table 2 summarize the findings and Appendix G reports full numbers. A sanity-check task (Naive) in which all models except XGBoost achieve near-perfect AUC is reported in Appendix B.

OC-Deterministic: bidirectional and recurrent models learn the offset efficiently.

When compliance requires detecting a lag-7 ordered key run, pretrained bidirectional encoders (BERT, RoBERTa) and small randomly initialized sequential models (Transformer, LSTM, RNNTransformer; 100K–400K parameters) reach $AUC \geq 0.997$.

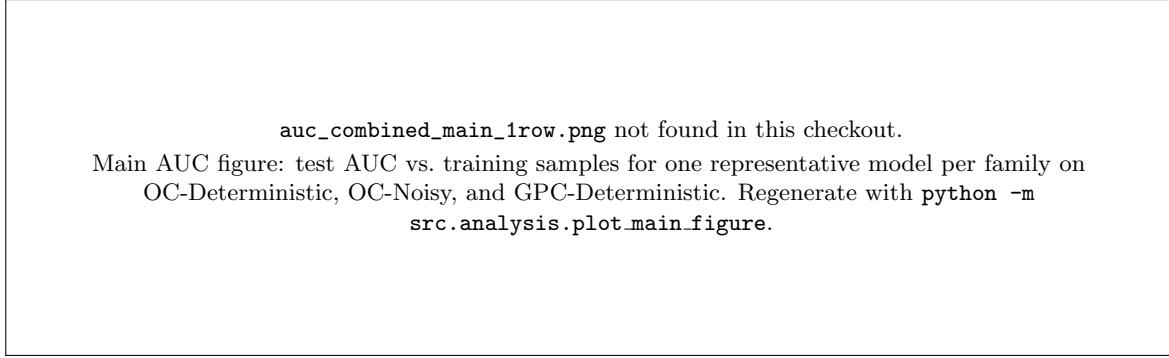


Figure 1. Test AUC across three synthetic datasets. For readability, we show one representative model per family; full results across all 13 models are reported in Appendix G. **OC-Deterministic:** bidirectional encoders, recurrent models, and larger causal decoders (8B) reach optimality, while smaller decoders (1B) plateau at ~ 0.85 . **OC-Noisy:** bidirectional models approach the optimal bound (0.67) with modest parameter counts, while decoder-only models require substantially more parameters (14B) to reach similar performance. In **GPC-Deterministic** all models fail the task, regardless of the number of parameters (we tested up to 70B decoder-only).

Decoder-only Llama-3.2-1B plateaus at $\text{AUC} \approx 0.85$, close to the contiguous 2-gram baseline of 0.82, while Llama-3.1-8B reaches $\text{AUC} = 0.995$. XGBoost on frozen Llama-3.2-1B mean-pooled embeddings also fails; this likely reflects a mismatch between pre-trained representations and our synthetic letter task rather than a property of the embedding architecture per se. The mechanism-identification audit of Section 5.1 shows that a 676-dimensional logistic regression on the λ -aware pair feature $g_{ab}^{(\lambda=7)}$ also reaches $\text{AUC} = 0.997$, and that a λ -agnostic 12,844-dimensional logistic regression recovers $\lambda = 7$ on its own. We therefore read the encoder-decoder gap on this task as an efficiency gap in discovering the right lagged-pair feature from raw-token supervision, rather than as evidence for global sequence integration.

OC-Noisy: scale unlocks autoregressive models. At $\pi = 0.3$ the theoretical upper bound is $\text{AUC}^* = 0.67$. Our custom Transformer and RN-Transformer reach this ceiling ($\text{AUC} = 0.671$), BERT reaches $\text{AUC} = 0.669$, and LSTM reaches $\text{AUC} = 0.665$. Smaller decoder-only models (Llama-3.2-1B, 8B) plateau near $\text{AUC} = 0.587$, close to the contiguous 2-gram baseline ($= 0.585$), but at sufficient scale Qwen-14B also approaches the ceiling. The mechanism-identification ladder again matches the ceiling with a linear model when λ is given (L_2 logistic regression on $g_{ab}^{(\lambda=7)}$, $\text{AUC} = 0.671$) and with a boosted tree when λ is not given (λ -agnostic XGBoost on stacked all-offset pair counts, $\text{AUC} = 0.667$). The λ -agnostic linear baseline underperforms the ceiling ($\text{AUC} = 0.632$): under label noise the right λ -block is not separable by a linear sparsity pattern and the top-weight lag on L_1 logistic regression drifts to $\lambda = 19$ (Appendix I). Decoder-only models can reach the ceiling at sufficient scale in our tested regime, but require substantially more

parameters than encoders to match what a boosted tree does with hand-crafted pair features.

GPC-Deterministic: a hidden-subset identification failure, not a modular-arithmetic failure. Every tested model is at chance ($\text{AUC} \approx 0.50$) on hidden-subset parity, including 70B-parameter LLMs, bidirectional transformers, and recurrent networks; the k -gram baselines also fail, consistent with Fu et al. [45]. This holds both in the main $|K| = 6$ regime and, for the core architectures, in the exactly balanced $|K| = 13$ regime (Appendix I). A decomposition (Appendix I.4) localizes the failure to joint hidden-subset identification and modular arithmetic under end-to-end supervision, rather than to modular arithmetic itself: a 64-unit MLP fed the per-position membership bits $b_t = \mathbf{1}\{X_t \in K\}$ (a 20-dim binary input that does not name the key letters) reaches $\text{AUC} = 0.997$ in under a minute, and $\text{AUC} = 1.000$ when fed the 6-dim per-key parity vector. Binary-vocabulary parity and induction-head circuits succeed on related tasks [20]; we reproduce one such protocol and locate our regime on the non-learnable side of a length-driven phase boundary (Appendix I.5). The negative result is therefore about end-to-end supervised learning of hidden-subset identification, not a general claim on parity learnability. Few-shot in-context evaluation of Llama-3.2-1B, Llama-3.1-8B, and Qwen2.5-14B fails on both OC variants and on GPC, indicating the main fine-tuning results are not an artifact of access mode (Appendix I.9).

5.1 Mechanism-identification audit and sensitivity sweep

To explain the drivers of model performance, we evaluate a hierarchy of seven feature sets across three simple classifiers (Appendix I). On both OC variants,

Table 2. Summary of model performance across datasets, using criteria from Section 4.3. ✓: full recovery ($AUC \approx AUC^*$); ~: partial recovery ($0.55 < AUC < AUC^*$); ×: no learning ($AUC \approx 0.50$). ★: most data-efficient models to reach the optimum. —: not tested on that dataset. All evaluations based on 100% training data.

Model	OC-Det.	OC-Noisy	GPC-Det.
k -gram baseline	~	~	×
XGBoost	~	×	×
BERT	★	★	×
RoBERTa	✓	✓	×
Llama-3.2-1B	~	~	×
Qwen3-4B-Think	~	~	×
Llama-3.1-8B	✓	~	×
Qwen2.5-14B	—	✓	—
Llama-3.1-70B	—	—	×
LSTM	✓	✓	×
Transformer	★	★	×
RNNTransformer*	✓	✓	×

*We call RNNTransformer a BiLSTM-Transformer.

a logistic regression trained solely on lag-aware aggregated pair counts perfectly matches the encoder ceiling, whereas simple content counts fall short (AUC 0.82 for deterministic, 0.58 for noisy). We contextualize this linear performance through two controls. First, a leave-one-rule-out experiment reveals that the linear model learns rule-specific weights rather than a transferable algorithm (within-rule AUC 1.00 vs. across-rule AUC 0.50; Appendix I.3). Second, even after matching on key-letter counts, lag-aware features retain a non-trivial predictive signal (within-group AUC 0.65 vs. chance 0.50 on OC-Noisy). Internally, attention analysis confirms that while BERT suppresses non-key positions, it does not preferentially attend to lag-connected key pairs (Appendix J). Finally, a sensitivity sweep over m and λ (Appendix H) preserves the same qualitative pattern: BERT and LSTM operate near the oracle ceiling, while Llama-3.2-1B remains consistently less efficient.

5.2 A length-based curriculum partially recovers the GPC mechanism

Hahn [25] identifies length-dependent difficulties for transformer parity learning (the key ingredient of our GPC mechanism), consistent with what we observe: small encoders learn parity at $n = 10$ but collapse to chance at $n = 20$ (Appendix I.5). We exploit this gap with a two-stage curriculum: train from scratch on $n = 10$, then warm-start on $n = 20$ with positional embeddings transferred where applicable. This recovers global-property-compliance for an LSTM and for a small randomly initialized Trans-

former, but not for pretrained BERT or Llama under our tested recipes (Appendix I.10). The bottleneck for shallow random-init architectures is therefore addressable by training-paradigm interventions, while pretrained models require additional study.

5.3 Real-world audit: limited gains can reflect non-decisive sequential information

Our empirical results demonstrate that for CKD→ESRD progression, the prediction task becomes heavily shortcut-dominated under a diagnosis-code representation. Unigrams alone capture most of the predictive signal ($AUC 0.823 \pm 0.015$), outperforming contiguous 2-grams (0.801 ± 0.016). Order-invariant baselines are similarly strong, with Logistic Regression and XGBoost reaching AUCs of 0.809 and 0.803, respectively. A Transformer achieves the peak ordered AUC (0.860), but most of this gain remains after randomly shuffling each patient’s sequence (AUC 0.844), while BiLSTM and BERT show no ordered-versus-shuffled gain. In this representation, the task can be solved by exploiting diagnosis identity, frequency, and co-occurrence: learning *what* diagnoses occur is largely sufficient, so learning *when* they occur is not decisive for performance. Full results in Appendix G.1.

6 Discussion and limitations

This work introduced a semantic-free diagnostic framework to test whether sequence models recover the intended latent mechanism or exploit simpler predictive shortcuts. The two mechanisms expose complementary regimes. In the OC mechanism, models can perform well by discovering lag-aware pair-count features; indeed, these features match the empirical ceiling reached by the best encoders. Thus, strong OC performance should be interpreted as efficient recovery of the relevant lagged statistic, not as evidence of global sequence integration. In the GPC mechanism, all models fail under standard end-to-end training, while the task becomes solvable once hidden membership information is provided. This localizes the difficulty to joint hidden-subset identification and global aggregation.

From the theoretical perspective, Theorem 3.3 is specific to the OC mechanism and does not extend to arbitrary order-sensitive labels. For example, if $\mathcal{A} = \{0, 1, 2, 3, 4\}$, $X_t \stackrel{iid}{\sim} \text{Uniform}(\mathcal{A})$, and $Y = \mathbf{1}\{(X_2 + 2X_3) \bmod 5 \in \{0, 1\}\}$, then $I(Y; X_t) = 0$ for every t , although Y depends on the ordered pair (X_2, X_3) . The theorem also guarantees only the existence of single-position leakage, but does not provide a theoretical lower bound on its magnitude. Theorem 3.5 should be read as a bounded-local

result.

The synthetic setting is intentionally narrow. It targets two capabilities: recovering a hidden positional offset from raw tokens, and jointly identifying a hidden subset while applying a global modular rule. It is not a general benchmark for long-range sequential learning. Empirically, we focus on one main regime per task, with sensitivity sweeps and ablations in the appendix; most models are tested up to 14B parameters, with the 70B model evaluated only on the GPC mechanism. The theoretical claims also rely on independent full-support sampling and should not be read as impossibility results for arbitrary real-world sequence distributions, transformers, or parity-like tasks in general.

Finally, our MIMIC-IV audit is limited to a single clinical case study. However, as recent work highlights [1], while true temporal dependencies certainly exist in clinical trajectories, they are neither ubiquitous nor trivial to isolate. Therefore, judging a sequence model’s architectural adequacy based solely on its raw performance on real-world sequential tasks requires extreme caution. Although our case study is not an exhaustive benchmark, it efficiently demonstrates a broader fundamental principle: apparent sequentiality in real data does not imply an absence of predictive shortcuts.

References

- [1] B. Edelman, H. Kim, and J. Skolnick. “Scalable Discovery and Validation of Order-Specific Electronic Health Record Event Trajectories for Interpretable Adverse-Outcome Risk Estimation”. In: *Journal of Biomedical Informatics* 177 (May 2026), p. 105008. ISSN: 1532-0464. DOI: [10.1016/j.jbi.2026.105008](https://doi.org/10.1016/j.jbi.2026.105008).
- [2] D. Placido, B. Yuan, J. X. Hjaltelin, C. Zheng, A. D. Haue, P. J. Chmura, C. Yuan, J. Kim, R. Umeton, G. Antell, A. Chowdhury, A. Franz, L. Brais, E. Andrews, D. S. Marks, A. Regev, S. Ayandeh, M. T. Brophy, N. V. Do, P. Kraft, B. M. Wolpin, M. H. Rosenthal, N. R. Fillmore, S. Brunak, and C. Sander. “A Deep Learning Algorithm to Predict Risk of Pancreatic Cancer from Disease Trajectories”. In: *Nature Medicine* 29.5 (2023), pp. 1113–1122. ISSN: 1546-170X. DOI: [10.1038/s41591-023-02332-5](https://doi.org/10.1038/s41591-023-02332-5). URL: <https://www.nature.com/articles/s41591-023-02332-5>.
- [3] T. Beaney, S. Jha, A. Alaa, A. Smith, J. Clarke, T. Woodcock, A. Majeed, P. Aylin, and M. Barahona. “Comparing Natural Language Processing Representations of Coded Disease Sequences for Prediction in Electronic Health Records”. In: *Journal of the American Medical Informatics Association* 31.7 (2024), pp. 1451–1462. ISSN: 1067-5027. DOI: [10.1093/jamia/ocae091](https://doi.org/10.1093/jamia/ocae091). URL: <https://academic.oup.com/jamia/article/31/7/1451/7667337>.
- [4] F. Pollington, S. C. Denaxas, K. Li, J. H. Thygesen, G. Lyrtatzopoulos, and B. White. “Evaluation of Trajectory Analysis for Disease Risk Assessment: A Scoping Review”. In: *Journal of the American Medical Informatics Association* 33.2 (2026), pp. 521–535. ISSN: 1527-974X. DOI: [10.1093/jamia/ocaf208](https://doi.org/10.1093/jamia/ocaf208). URL: <https://doi.org/10.1093/jamia/ocaf208>.
- [5] S. Kim, H. Kang, K. Kim, J. Kim, D. Kim, M. Yang, K. Oh, J. McAuley, and C. Park. “Lost in Sequence: Do Large Language Models Understand Sequential Recommendation?” In: *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2*. ACM, Aug. 2025, pp. 1160–1171. ISBN: 979-8-4007-1454-2. DOI: [10.1145/3711896.3737035](https://doi.org/10.1145/3711896.3737035).
- [6] Y. Li, S. Rao, J. R. A. Solares, et al. “BEHRT: Transformer for Electronic Health Records”. In: *Scientific Reports* 10 (2020), p. 7155.
- [7] L. Rasmy, Y. Xiang, Z. Xie, C. Tao, and D. Zhi. “Med-BERT: Pretrained Contextualized Embeddings on Large-Scale Structured Electronic Health Records for Disease Prediction”. In: *npj Digital Medicine* 4 (2021), p. 86.
- [8] G. Savcicens, T. Eliassi-Rad, L. K. Hansen, L. H. Mortensen, L. Lilleholt, A. Rogers, I. Zettler, and S. Lehmann. “Using Sequences of Life-Events to Predict Human Lives”. In: *Nature Computational Science* 4.1 (Dec. 2023), pp. 43–56. ISSN: 2662-8457. DOI: [10.1038/s43588-023-00573-5](https://doi.org/10.1038/s43588-023-00573-5). URL: <http://dx.doi.org/10.1038/s43588-023-00573-5>.
- [9] B. Fatemi, M. Kazemi, A. Tsitsulin, K. Malkan, J. Yim, J. Palowitch, S. Seo, J. Halcrow, and B. Perozzi. *Test of Time: A Benchmark for Evaluating LLMs on Temporal Reasoning*. 2024. arXiv: [2406.09170](https://arxiv.org/abs/2406.09170). URL: <https://arxiv.org/abs/2406.09170>.
- [10] A. Shmatko, A. W. Jung, K. Gaurav, S. Brunak, L. H. Mortensen, E. Birney, T. Fitzgerald, and M. Gerstung. “Learning the Natural History of Human Disease with Generative Transformers”. In: *Nature* 647.8088 (Nov. 2025), pp. 248–256. ISSN: 1476-4687. DOI: [10.1038/s41586-025-09529-3](https://doi.org/10.1038/s41586-025-09529-3).
- [11] S. Bedi, Y. Jiang, P. Chung, S. Koyejo, and N. Shah. “Fidelity of Medical Reasoning in Large Language Models”. In: *JAMA Network Open* 8.8 (Aug. 2025), e2526021. ISSN: 2574-

3805. DOI: [10.1001/jamanetworkopen.2025.26021](https://doi.org/10.1001/jamanetworkopen.2025.26021). [22] D. Kaiser, A. Frigessi, A. Ramezani-Kebrya, and B. Ricaud. “CogniLoad: A Synthetic Natural Language Reasoning Benchmark with Tunable Length, Intrinsic Difficulty, and Distractor Density”. In: *arXiv preprint arXiv:2509.18458* (2025).
- [12] T. Nguyen. *Understanding Transformers via N-gram Statistics*. 2024. arXiv: [2407.12034](https://arxiv.org/abs/2407.12034). URL: <https://arxiv.org/abs/2407.12034>.
- [13] M. Du, F. Li, G. Zheng, and V. Srikumar. “DeepLog: Anomaly Detection and Diagnosis from System Logs through Deep Learning”. In: *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS ’17)*. ACM, 2017, pp. 1285–1298. DOI: [10.1145/3133956.3134015](https://doi.org/10.1145/3133956.3134015). URL: <https://doi.org/10.1145/3133956.3134015>.
- [14] I. Teinmaa, M. Dumas, M. La Rosa, and F. M. Maggi. “Outcome-Oriented Predictive Process Monitoring: Review and Benchmark”. In: *ACM Transactions on Knowledge Discovery from Data* 13.2 (2019), 17:1–17:57. ISSN: 1556-4681. DOI: [10.1145/3301300](https://doi.org/10.1145/3301300). URL: <https://doi.org/10.1145/3301300>.
- [15] H. Cui, A. Unell, B. Chen, J. A. Fries, E. Alsentzer, S. Koyejo, and N. Shah. *Timer: Temporal Instruction Modeling and Evaluation for Longitudinal Clinical Records*. 2025. arXiv: [2503.04176](https://arxiv.org/abs/2503.04176). URL: <https://arxiv.org/abs/2503.04176>.
- [16] A. Klenitskiy, A. Volodkevich, A. Pembek, and A. Vasilev. “Does It Look Sequential? An Analysis of Datasets for Evaluation of Sequential Recommendations”. In: *18th ACM Conference on Recommender Systems*. Oct. 2024, pp. 1067–1072. DOI: [10.1145/3640457.3688195](https://doi.org/10.1145/3640457.3688195).
- [17] R. T. McCoy, E. Pavlick, and T. Linzen. “Right for the Wrong Reasons: Diagnosing Syntactic Heuristics in Natural Language Inference”. In: *Proceedings of ACL*. 2019, pp. 3428–3448.
- [18] R. Geirhos, J.-H. Jacobsen, C. Michaelis, R. Zemel, W. Brendel, M. Bethge, and F. A. Wichmann. “Shortcut Learning in Deep Neural Networks”. In: *Nature Machine Intelligence* 2.11 (2020), pp. 665–673.
- [19] I. Mirzadeh, K. Alizadeh, H. Shahrokhi, O. Tuzel, S. Bengio, and M. Farajtabar. “GSM-Symbolic: Understanding the Limitations of Mathematical Reasoning in Large Language Models”. In: *arXiv preprint arXiv:2410.05229* (2024).
- [20] C. Olsson, N. Elhage, N. Nanda, et al. “In-context Learning and Induction Heads”. In: *arXiv preprint arXiv:2209.11895* (2022).
- [21] A. Gu and T. Dao. *Mamba: Linear-Time Sequence Modeling with Selective State Spaces*. 2024. arXiv: [2312.00752](https://arxiv.org/abs/2312.00752). URL: <http://arxiv.org/abs/2312.00752>.
- [22] D. Kaiser, A. Frigessi, A. Ramezani-Kebrya, and B. Ricaud. “CogniLoad: A Synthetic Natural Language Reasoning Benchmark with Tunable Length, Intrinsic Difficulty, and Distractor Density”. In: *arXiv preprint arXiv:2509.18458* (2025).
- [23] M. Ramezanali, M. Vazifeh, and P. Santi. *seqBench: A Tunable Benchmark to Quantify Sequential Reasoning Limits of LLMs*. Sept. 2025.
- [24] A. E. W. Johnson, L. Bulgarelli, L. Shen, A. Gayles, A. Shammout, S. Horng, T. J. Pollard, S. Hao, B. Moody, B. Gow, L.-w. H. Lehman, L. A. Celi, and R. G. Mark. “MIMIC-IV, a Freely Accessible Electronic Health Record Dataset”. In: *Scientific Data* 10.1 (2023), p. 1. DOI: [10.1038/s41597-022-01899-x](https://doi.org/10.1038/s41597-022-01899-x).
- [25] M. Hahn. “Theoretical Limitations of Self-Attention in Neural Sequence Models”. In: *Transactions of the Association for Computational Linguistics* 8 (2020), pp. 156–171. DOI: [10.1162/tac1_a_00306](https://doi.org/10.1162/tac1_a_00306). URL: <https://aclanthology.org/2020.tac1-1.11/>.
- [26] H. Touvron, T. Lavril, G. Izacard, et al. “LLaMA: Open and Efficient Foundation Language Models”. In: *arXiv preprint arXiv:2302.13971* (2023).
- [27] A. Grattafiori, A. Dubey, et al. “The Llama 3 Herd of Models”. In: *arXiv preprint arXiv:2407.21783* (2024).
- [28] Qwen Team. *Qwen3 Technical Report*. 2025. arXiv: [2505.09388](https://arxiv.org/abs/2505.09388). URL: <https://arxiv.org/abs/2505.09388>.
- [29] Qwen Team. *Qwen2.5: A Party of Foundation Models*. Sept. 2024. URL: <https://qwenlm.github.io/blog/qwen2.5/>.
- [30] A. Yang, B. Yang, B. Hui, B. Zheng, B. Yu, C. Zhou, C. Li, C. Li, D. Liu, F. Huang, G. Dong, H. Wei, H. Lin, J. Tang, J. Wang, J. Yang, J. Tu, J. Zhang, J. Ma, J. Xu, J. Zhou, J. Bai, J. He, J. Lin, K. Dang, K. Lu, K. Chen, K. Yang, M. Li, M. Xue, N. Ni, P. Zhang, P. Wang, R. Peng, R. Men, R. Gao, R. Lin, S. Wang, S. Bai, S. Tan, T. Zhu, T. Li, T. Liu, W. Ge, X. Deng, X. Zhou, X. Ren, X. Zhang, X. Wei, X. Ren, Y. Fan, Y. Yao, Y. Zhang, Y. Wan, Y. Chu, Y. Liu, Z. Cui, Z. Zhang, and Z. Fan. “Qwen2 Technical Report”. In: *arXiv preprint arXiv:2407.10671* (2024).
- [31] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova. “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding”. In: *Proceedings of NAACL*. 2019, pp. 4171–4186.

- [32] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov. *RoBERTa: A Robustly Optimized BERT Pretraining Approach*. 2019. DOI: [10.48550/arXiv.1907.11692](https://arxiv.org/abs/1907.11692). arXiv: [1907.11692](https://arxiv.org/abs/1907.11692) [cs.CL]. URL: <https://arxiv.org/abs/1907.11692>.
- [33] S. Hochreiter and J. Schmidhuber. “Long Short-Term Memory”. In: *Neural Computation* 9.8 (1997), pp. 1735–1780.
- [34] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. “Attention is All You Need”. In: *Advances in Neural Information Processing Systems* 30 (2017).
- [35] A. Graves and J. Schmidhuber. “Framewise Phoneme Classification with Bidirectional LSTM and Other Neural Network Architectures”. In: *Neural Networks* 18.5-6 (2005), pp. 602–610.
- [36] T. Chen and C. Guestrin. “XGBoost: A Scalable Tree Boosting System”. In: *Proceedings of KDD*. 2016, pp. 785–794.
- [37] T. Do, B. D. S. Gurung, S. Aryal, A. Khanal, S. Chataut, V. Gadhamshetty, C. Lushbough, and E. Z. Gnimpieba. “Utilizing XGBoost for the Prediction of Material Corrosion Rates from Embedded Tabular Data Using Large Language Model”. In: *2023 IEEE International Conference on Bioinformatics and Biomedicine (BIBM)*. 2023, pp. 4497–4499. DOI: [10.1109/BIBM58861.2023.10385544](https://doi.org/10.1109/BIBM58861.2023.10385544).
- [38] A. Alshraideh, B. Al Fayoumi, B. M. Alshraideh, and M. Alshraideh. “Hybrid AI Framework for the Early Detection of Heart Failure: Integrating Traditional Machine Learning and Generative Language Models with Clinical Data”. In: *Cureus* 17.6 (2025), e85638.
- [39] I. Loshchilov and F. Hutter. *Decoupled Weight Decay Regularization*. 2019. arXiv: [1711.05101](https://arxiv.org/abs/1711.05101). URL: <https://arxiv.org/abs/1711.05101>.
- [40] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen. *LoRA: Low-Rank Adaptation of Large Language Models*. 2021. arXiv: [2106.09685](https://arxiv.org/abs/2106.09685). URL: <https://arxiv.org/abs/2106.09685>.
- [41] A. Johnson, L. Bulgarelli, T. Pollard, B. Gow, B. Moody, S. Horng, L. A. Celi, and R. Mark. *MIMIC-IV (version 3.1)*. PhysioNet. Oct. 2024. DOI: [10.13026/kpb9-mt58](https://doi.org/10.13026/kpb9-mt58). URL: <https://doi.org/10.13026/kpb9-mt58>.

Table .3. Purpose of appendix analyses.

Appendix	Purpose
A	Real-world motivation
B	Naive Sanity Check
C	Ordering Mechanism Example
D	Contiguous k-gram shortcut analysis
E	Proofs of local leakage and parity purity
F	Reproducibility and model configurations
G	Full numerical results
H	Sensitivity to hidden-subset size and lag
I.1–I.4	Mechanism-identification audit
I.5–I.8	Parity robustness checks
I.9	In-context access-mode control
I.10	Curriculum control for parity learnability
J	Exploratory attention inspection, not primary evidence

- [42] K. Sumida and C. P. Kovesdy. “Disease Trajectories Before ESRD: Implications for Clinical Management”. In: *Seminars in Nephrology* 37.2 (Mar. 2017), pp. 132–143. ISSN: 0270-9295. DOI: [10.1016/j.semnephrol.2016.12.003](https://doi.org/10.1016/j.semnephrol.2016.12.003).
- [43] Kidney Disease: Improving Global Outcomes (KDIGO) CKD Work Group. “KDIGO 2024 Clinical Practice Guideline for the Evaluation and Management of Chronic Kidney Disease”. In: *Kidney International* 105.4S (2024), S117–S314. DOI: [10.1016/j.kint.2023.10.018](https://doi.org/10.1016/j.kint.2023.10.018). URL: <https://doi.org/10.1016/j.kint.2023.10.018>.
- [44] Agency for Healthcare Research and Quality. *HCUP Clinical Classifications Software (CCS) for ICD-9-CM*. Healthcare Cost and Utilization Project (HCUP). Sept. 2025. URL: <https://hcup-us.ahrq.gov/toolssoftware/ccs/ccs.jsp> (visited on 07/31/2026).
- [45] T. Fu, R. Ferrando, J. Conde, C. Arriaga, and P. Reviriego. *Why Do Large Language Models (LLMs) Struggle to Count Letters?* 2024. arXiv: [2412.18626](https://arxiv.org/abs/2412.18626). URL: <https://arxiv.org/abs/2412.18626>.

Appendix roadmap. Appendices A–F provide task examples, proofs, and reproducibility details. Appendix G reports full model results. Appendix H tests robustness of the compliance findings across subset size and lag. Appendix I contains the mechanism-identification audits and additional parity controls used to rule out alternative explanations. Appendix J provides an exploratory attention inspection; we do not use it as primary evidence for mechanism attribution.

A Real-world trajectory motivation and synthetic abstraction

Consider adverse-outcome prediction from longitudinal EHRs. Edelman et al. [1] study order-specific

trajectories of the form $A \rightarrow B \rightarrow C$: two upstream events A and B define an ordered exposure, and C is a downstream adverse outcome. Their comparator design asks whether the ordered exposure $A \rightarrow B$ changes risk relative to cohorts that preserve part of the same event content while altering temporal structure, including B with no prior A , the reverse ordering $B \rightarrow A$, and an unordered AB cohort. One validated example is diabetes mellitus $\rightarrow$ visual disturbance for downstream myocardial infarction; at the five-year horizon, the ordered trajectory has elevated risk relative to both the primary comparator and the reverse-order comparator. We use this only as a motivating analogy: the association is prognostic rather than causal, and our benchmark does not model clinical outcomes, censoring, observation processes, or risk calibration. In particular, the clinical C is a future outcome, whereas Y^* in our compliance task is a synthetic label indicating whether a hidden ordered motif is present.

Our synthetic tasks abstract only the mechanism-attribution problem exposed by such studies. In real EHRs, a model’s success may come from event identity, disease semantics, code frequency, recency, care-utilization patterns, or local co-occurrence rather than from the intended temporal rule. We therefore replace event types by letters and remove semantic content: for exposition, diabetes mellitus can be denoted D and visual disturbance V , but these mnemonic choices are not used in the benchmark. In the experiments, the relevant subset and the ordering rule κ are hidden and randomized, so a model cannot exploit medical meaning, alphabetical order, or pre-existing associations.

As a toy walk-through, let a patient history be represented by

$X = \text{BXDQAVZHELNPTKDSRAJ},$

with length $n = 20$. Suppose the hidden key subset is $S = \{D, V\}$, with $\kappa(D) < \kappa(V)$, and let $\lambda = 3$. Then positions 3 and 6 form a lag- λ ordered key pair, $D \rightarrow V$, so the sequence is compliant. The lag λ is only a discrete abstraction of temporal spacing; it is not intended to represent a clinical time interval such as days or years. A rule-based solution would require recovering which letters matter, their ordering, and the relevant lag. The purpose of our audit is to determine whether neural models recover this hidden structure from raw tokens or whether their performance is already explained by simpler feature families such as unordered counts, contiguous k -grams, or lag-aware pair counts.

B Degenerate local sanity check: Naive

We include **Naive** only as a pipeline sanity check, not as a core benchmark. Here $m = \ell$, $\lambda = 1$, and κ is the natural alphabetical order, so $Y^* = 1$ iff the entire sequence is non-decreasing. This construction makes the label locally detectable: descending adjacent bigrams occur only in negatives, and a contiguous 2-gram classifier reaches $\text{AUC} \approx 0.999$. Consistent with this, all neural sequential models reach near-perfect AUC with $\geq 40K$ training examples, while XGBoost without explicit adjacent-pair features does not (Table G.1). We therefore use Naive only to verify the training and evaluation pipeline, and exclude it from the main benchmark.

C Compliance Mechanisms

D Contiguous k -gram shortcut analysis

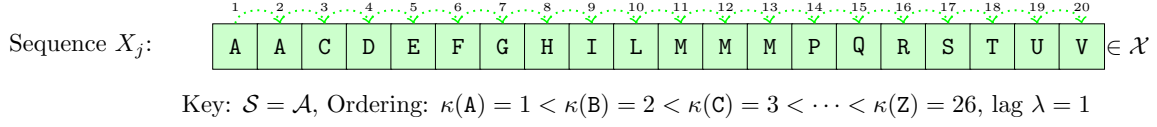
This appendix quantifies how much outcome signal is recoverable from contiguous local patterns alone. A k -gram is a contiguous subsequence $(X_t, \dots, X_{t+k-1})$. For each k -gram g observed in the training set, we estimate $\hat{p}(g) = \#(g, Y = 1) / \#(g)$. For a test sequence X , we score

$$s_k(X) = \frac{1}{n - k + 1} \sum_{t=1}^{n-k+1} \hat{p}(X_t, \dots, X_{t+k-1}).$$

Unseen k -grams are assigned the training-set positive rate. We report AUC, rather than threshold-dependent metrics, because AUC is the primary metric for latent-rule recovery.

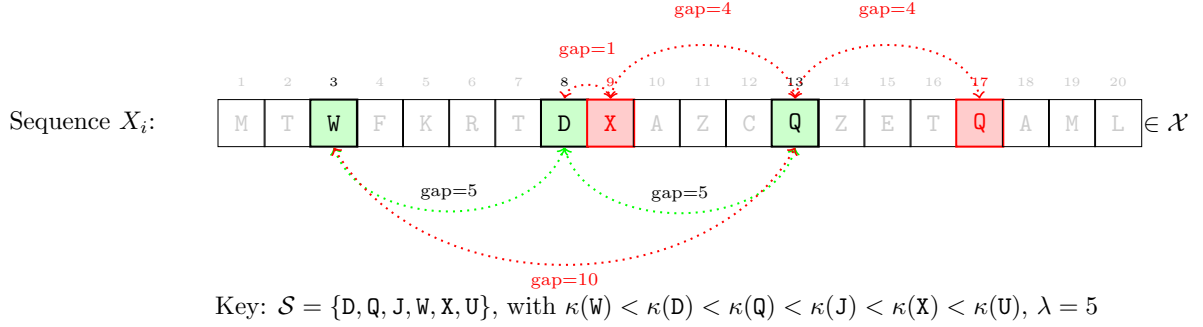
Table D.1 shows that contiguous k -grams capture nontrivial local signal in the compliance tasks but do not reach the neural or lag-aware pair-count ceiling. In OC-Deterministic, the best contiguous k -gram score is around 0.82 AUC; in OC-Noisy it is around 0.59, near the smaller decoder-only models. In contrast, all contiguous k -gram scores remain at chance on hidden-subset parity. Coverage collapses for $k \geq 5$, so longer contiguous patterns are too sparse to provide reliable generalization in this regime.

Table D.2 separates content-only, contiguous-local, and lag-aware signal. The key finding is that contiguous k -grams explain part of the compliance performance but not the ceiling: the ceiling is reached only after exposing the λ -aware pair-count feature.



The subsequence consisting on the whole sequence satisfies both lag $\lambda=1$ and the ordering key $\kappa(\mathcal{S}) \equiv \mathcal{A}$.
 $\Rightarrow$ Ordered (i.e., $Y^* = 1$)

Figure C.1. Sequence classification example in the **Naive** sanity check. Key letters $\mathcal{S} = \mathcal{A}$ (i.e., all letters). Arrows show gaps between key letters. In green letters (and gaps) that do have lag-connections with other key letters and that satisfy the ordering $\kappa(\mathcal{S}) \equiv \mathcal{A}$. Note: in this example $n = 20$, $m = 26$.



The subsequence W-D-Q satisfies both lag $\lambda=5$ and the ordering key $\kappa(\mathcal{S}) \Rightarrow$ Ordered (i.e., $Y^* = 1$)

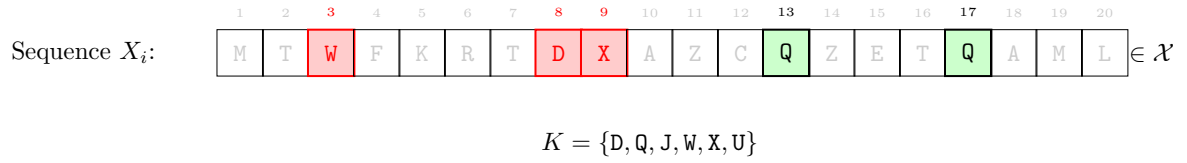
Figure C.2. Sequence classification with **OC** mechanism. Key letters $\mathcal{S} = \{D, Q, J, W, X, U\}$ are extracted from the full sequence (non-key in gray). Arrows indicate gaps. Red: letters/gaps without lag-connections to other keys. Green: letters/gaps with lag-connections satisfying ordering $\kappa(\mathcal{S})$. Here $n = 20$, $m = 6$.

Table D.1. Contiguous k -gram baseline. Coverage is the fraction of test k -grams observed in the training set. OC and GPC mechanisms share the same i.i.d. sequence proposal, so coverage is common across these tasks.

k	Coverage	OC-Deterministic AUC	OC-Noisy AUC	GPC-Deterministic AUC
1	100.0%	.813	.585	.502
2	100.0%	.819	.586	.498
3	100.0%	.818	.576	.497
4	100.0%	.781	.534	.499
5	41.7%	.617	.508	.497
6	1.9%	.524	.499	.501
7	0.1%	.501	.501	.500

Table D.2. Signal decomposition by feature family, measured by test AUC.

Method	OC-Deterministic	OC-Noisy	GPC-Deterministic
Chance	.500	.500	.500
BoE-key counts	.774	.574	.503
BoE-XGB counts	.816	.584	.503
Best contiguous k -gram	.819	.586	.502
λ -aware pair LR	.997	.671	—
Best neural model	.999	.671	.503
Oracle AUC	1.000	.670	1.000



An odd number of letters belonging to K appear in an even manner (including 0) in the sequence $\Rightarrow Y^* = 0$

Figure C.3. Sequence classification example for the **GPC** mechanism. The parity-relevant subset is $K = \{D, Q, J, W, X, U\}$. The parity criterion requires *an even number of unique* key letter to appear an **even** number of times (including 0, which is even). In green: key letters with even count. In red: key letters with odd count. Note: that also J and U have even count, including zero, in the sequence satisfying Definition 3.4. So we have in total 3 letters in K with an even count, and since 3 is not even the sequence is labeled as $Y^* = 0$.

E Proofs

Theorem E.1 (Optimal Performance under Label Noise). *Under symmetric label noise with parameter $\pi \in [0, 0.5]$ and compliance rate ρ (i.e., $\mathbb{P}(Y^* = 1) = \rho$, and $\mathbb{P}(Y = 1 | Y^* = 1) = 1 - \pi$), a classifier with perfect access to compliance status (oracle) achieves:*

$$\text{AUC}^* = \frac{1}{2} + \frac{1}{2}(A - C) \stackrel{\rho=0.5}{=} 1 - \pi$$

where

$$A = \frac{(1 - \pi)\rho}{\rho(1 - \pi) + (1 - \rho)\pi}, \quad C = \frac{\pi\rho}{\rho\pi + (1 - \rho)(1 - \pi)}.$$

Similarly, $F_1^* = 2(1 - \pi)\rho / (\pi + 2\rho(1 - \pi)) \stackrel{\rho=0.5}{=} 1 - \pi$. Per dataset, this gives $\text{AUC}^* = F_1^* = 1.000$ for Naive sanity check, OC-Deterministic, and GPC-Deterministic (all with $\pi = 0$), and $\text{AUC}^* = 0.670$, $F_1^* = 0.577$, $\text{Precision}^* = 0.700$, $\text{Recall}^* = 0.491$ for OC-Noisy ($\pi = 0.3$, $\rho = 0.293$).

Proof of Theorem E.1. Under symmetric label noise,

$$P(Y = 1 | Y^* = 1) = 1 - \pi, \quad P(Y = 1 | Y^* = 0) = \pi. \text{ Therefore:}$$

Because $\pi < 1/2$, the posterior $P(Y = 1 | Y^*)$ is strictly increasing in Y^* , so any Bayes-optimal ranking is equivalent to scoring by $s = Y^*$. We therefore compute the AUC of the binary oracle score $s \in \{0, 1\}$.

$$\text{AUC} = P(s_+ > s_-) + \frac{1}{2}P(s_+ = s_-) \quad (2)$$

where s_+ is the score for a random instance with $Y = 1$ and s_- is the score for a random instance with $Y = 0$.

The Joint Distribution is:

$$P(Y = 1, Y^* = 1) = (1 - \pi)\rho \quad (3)$$

$$P(Y = 1, Y^* = 0) = \pi(1 - \rho) \quad (4)$$

$$P(Y = 0, Y^* = 1) = \pi\rho \quad (5)$$

$$P(Y = 0, Y^* = 0) = (1 - \pi)(1 - \rho) \quad (6)$$

Marginal Distributions:

$$P(Y = 1) = \rho(1 - \pi) + (1 - \rho)\pi \quad (7)$$

$$P(Y = 0) = \rho\pi + (1 - \rho)(1 - \pi) \quad (8)$$

Note that $P(Y = 1) + P(Y = 0) = 1$.

For the conditional score distributions, define:

$$A = P(s = 1 | Y = 1) = \frac{(1 - \pi)\rho}{\rho(1 - \pi) + (1 - \rho)\pi} \quad (9)$$

$$B = P(s = 0 | Y = 1) = \frac{\pi(1 - \rho)}{\rho(1 - \pi) + (1 - \rho)\pi} \quad (10)$$

$$C = P(s = 1 | Y = 0) = \frac{\pi\rho}{\rho\pi + (1 - \rho)(1 - \pi)} \quad (11)$$

$$D = P(s = 0 | Y = 0) = \frac{(1 - \pi)(1 - \rho)}{\rho\pi + (1 - \rho)(1 - \pi)} \quad (12)$$

Note that $A + B = 1$ and $C + D = 1$.

AUC Calculation:

$$\text{AUC} = A \cdot D + \frac{1}{2}(A \cdot C + B \cdot D) \quad (13)$$

$$= A(1 - C) + \frac{1}{2}A \cdot C + \frac{1}{2}(1 - A)(1 - C) \quad (14)$$

$$= A - AC + \frac{1}{2}AC + \frac{1}{2} - \frac{1}{2}A - \frac{1}{2}C + \frac{1}{2}AC \quad (15)$$

$$= \frac{1}{2} + \frac{1}{2}(A - C) \quad (16)$$

Simplification for balanced classes:

For $\rho = 0.5$:

$$A = \frac{0.5(1 - \pi)}{0.5(1 - \pi) + 0.5\pi} = \frac{0.5(1 - \pi)}{0.5} = 1 - \pi \quad (17)$$

$$C = \frac{0.5\pi}{0.5\pi + 0.5(1 - \pi)} = \frac{0.5\pi}{0.5} = \pi \quad (18)$$

$$\text{AUC} = \frac{1}{2} + \frac{1}{2}[(1 - \pi) - \pi] = \frac{1}{2} + \frac{1}{2}(1 - 2\pi) = 1 - \pi \quad (19)$$

Alternatively, we can use the following expression.

Define $X = \rho(1 - \pi) + (1 - \rho)\pi$ and $Y = \rho\pi + (1 - \rho)(1 - \pi)$.

The numerator of $(A - C)$ can be computed as:

$$(A - C) \cdot XY = (1 - \pi)\rho \cdot Y - \pi\rho \cdot X \quad (20)$$

$$= \rho[(1 - \pi)Y - \pi X] \quad (21)$$

$$= \rho[(1 - \pi)(\rho\pi + (1 - \rho)(1 - \pi)) - \pi(\rho(1 - \pi) + (1 - \rho)\pi)] \quad (22)$$

$$= \rho[(1 - \pi)^2(1 - \rho) - \pi^2(1 - \rho)] \quad (23)$$

$$= \rho(1 - \rho)[(1 - \pi)^2 - \pi^2] \quad (24)$$

$$= \rho(1 - \rho)[(1 - \pi)^2 - \pi^2] \quad (25)$$

$$= \rho(1 - \rho)(1 - 2\pi) \quad (26)$$

Note: $(1 - \pi)^2 - \pi^2 = 1 - 2\pi + \pi^2 - \pi^2 = 1 - 2\pi$.

Therefore:

$$A - C = \frac{\rho(1 - \rho)(1 - 2\pi)}{XY} \quad (27)$$

and

$$\text{AUC} = \frac{1}{2} + \frac{\rho(1 - \rho)(1 - 2\pi)}{2XY} \quad (28)$$

□

For the thresholded oracle classifier $\hat{Y} = Y^*$, we now compute the resulting F1 score with respect to the observed labels.

Since we evaluate performance against the stochastic observed labels Y , the confusion matrix elements are:

$$\begin{aligned} 1137 \quad TP &= P(\hat{Y} = 1 \cap Y = 1) = P(Y^* = 1 \cap Y = 1) \\ 1138 \quad &= P(Y = 1 \mid Y^* = 1) \cdot P(Y^* = 1) = (1 - \pi)\rho \end{aligned} \quad (29)$$

$$\begin{aligned} 1139 \quad FP &= P(\hat{Y} = 1 \cap Y = 0) = P(Y^* = 1 \cap Y = 0) \\ 1140 \quad &= P(Y = 0 \mid Y^* = 1) \cdot P(Y^* = 1) = \pi\rho \end{aligned} \quad (30)$$

$$\begin{aligned} 1141 \quad FN &= P(\hat{Y} = 0 \cap Y = 1) = P(Y^* = 0 \cap Y = 1) \\ 1142 \quad &= P(Y = 1 \mid Y^* = 0) \cdot P(Y^* = 0) = \pi(1 - \rho) \end{aligned} \quad (31)$$

$$\begin{aligned} 1143 \quad TN &= P(\hat{Y} = 0 \cap Y = 0) = P(Y^* = 0 \cap Y = 0) \\ 1144 \quad &= P(Y = 0 \mid Y^* = 0) \cdot P(Y^* = 0) = (1 - \pi)(1 - \rho) \end{aligned} \quad (32)$$

For the precision, by definition:

$$\begin{aligned} 1146 \quad \text{Precision} &= \frac{TP}{TP + FP} = \frac{(1 - \pi)\rho}{(1 - \pi)\rho + \pi\rho} \\ 1147 \quad &= \frac{(1 - \pi)\rho}{\rho[(1 - \pi) + \pi]} = \frac{(1 - \pi)\rho}{\rho} = 1 - \pi \end{aligned} \quad (33)$$

Observation: Precision is independent of the class distribution ρ and depends only on the noise rate π .

For the recall, by definition:

$$\begin{aligned} 1152 \quad \text{Recall} &= \frac{TP}{TP + FN} = \frac{(1 - \pi)\rho}{(1 - \pi)\rho + \pi(1 - \rho)} \\ 1153 \quad &= \frac{(1 - \pi)\rho}{\rho - \pi\rho + \pi - \pi\rho} = \frac{(1 - \pi)\rho}{\rho(1 - 2\pi) + \pi} \end{aligned} \quad (34)$$

Verification for $\rho = 0.5$:

$$\begin{aligned} 1155 \quad \text{Recall} &= \frac{(1 - \pi) \cdot 0.5}{0.5(1 - 2\pi) + \pi} = \frac{0.5(1 - \pi)}{0.5 - \pi + \pi} \\ &= \frac{0.5(1 - \pi)}{0.5} = 1 - \pi \end{aligned} \quad (35)$$

Thus, for balanced classes, precision = recall = $1 - \pi$.

For F1, we use the harmonic mean of precision and recall:

$$1160 \quad F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \quad (36)$$

Substituting our expressions:

$$\begin{aligned} 1162 \quad F_1 &= \frac{2(1 - \pi) \cdot \frac{(1 - \pi)\rho}{\rho(1 - 2\pi) + \pi}}{(1 - \pi) + \frac{(1 - \pi)\rho}{\rho(1 - 2\pi) + \pi}} \\ 1163 \quad &= \frac{2(1 - \pi)^2\rho}{(1 - \pi)[\rho(1 - 2\pi) + \pi] + (1 - \pi)\rho} \end{aligned} \quad (37)$$

Simplifying the denominator:

$$\begin{aligned} 1165 \quad &(1 - \pi)[\rho(1 - 2\pi) + \pi] + (1 - \pi)\rho \\ 1166 \quad &= (1 - \pi)\rho(1 - 2\pi) + \pi(1 - \pi) + (1 - \pi)\rho \\ 1167 \quad &= (1 - \pi)\rho[(1 - 2\pi) + 1] + \pi(1 - \pi) \\ 1168 \quad &= (1 - \pi)\rho(2 - 2\pi) + \pi(1 - \pi) \\ 1169 \quad &= (1 - \pi)[2\rho(1 - \pi) + \pi] \end{aligned} \quad (38)$$

Therefore:

$$\begin{aligned} 1170 \quad F_1 &= \frac{2(1 - \pi)^2\rho}{(1 - \pi)[2\rho(1 - \pi) + \pi]} = \frac{2(1 - \pi)\rho}{2\rho(1 - \pi) + \pi} \\ 1171 \quad &= \frac{2(1 - \pi)\rho}{\pi + 2\rho(1 - \pi)} \end{aligned} \quad (39) \quad 1172$$

Verification for $\rho = 0.5$:

$$\begin{aligned} 1173 \quad F_1 &= \frac{2(1 - \pi) \cdot 0.5}{\pi + 2 \cdot 0.5 \cdot (1 - \pi)} \\ 1174 \quad &= \frac{1 - \pi}{\pi + (1 - \pi)} = \frac{1 - \pi}{1} = 1 - \pi \end{aligned} \quad (40) \quad 1175$$

□

E.1 Proof of Theorem 3.3

Proof. Let $s_{\min} \in \mathcal{S}$ satisfy $\kappa(s_{\min}) = 1$. We first show that the map

$$a \mapsto P(Y^* = 1 \mid X_1 = a) \quad 1179$$

is not constant on $\mathcal{A}$.

Case 1: $\mathcal{S} \subsetneq \mathcal{A}$. Choose $a_0 \in \mathcal{A} \setminus \mathcal{S}$. Because the coordinates of X are independent and have full support, both events $\{X_1 = s_{\min}\}$ and $\{X_1 = a_0\}$ have positive probability, and conditioning on X_1 does not change the joint law of $(X_2, \dots, X_n)$.

Let $Z_2, \dots, Z_n$ be independent random variables with $Z_t \sim \mathcal{L}(X_t)$, and define

$$\tilde{X} := (s_{\min}, Z_2, \dots, Z_n), \quad \tilde{X}' := (a_0, Z_2, \dots, Z_n). \quad 1188$$

Then

$$\tilde{X} \stackrel{d}{=} (X \mid X_1 = s_{\min}), \quad \tilde{X}' \stackrel{d}{=} (X \mid X_1 = a_0). \quad 1190$$

Let

$$C := \{\tilde{X} \text{ is compliant}\}, \quad C' := \{\tilde{X}' \text{ is compliant}\} \quad 1192$$

Since $\tilde{X}'_1 = a_0 \notin \mathcal{S}$, any ordered maximal λ -run in $\tilde{X}'$ must avoid position 1. If such a run does not start at $1 + \lambda$, the same maximal run is present in $\tilde{X}$. If it starts at $1 + \lambda$, then in $\tilde{X}$ it is extended to the left by $s_{\min}$, and the extended maximal run remains ordered because $\kappa(s_{\min}) = 1$ is the minimum rank. Hence every compliant witness in $\tilde{X}'$ induces a compliant witness in $\tilde{X}$, so

$$C' \subseteq C. \quad 1201$$

Now define the event

$$E := \{Z_{1+\lambda} \in \mathcal{S}\} \cap \bigcap_{t \in \{2, \dots, n\} \setminus \{1+\lambda\}} \{Z_t \notin \mathcal{S}\}. \quad 1203$$

Because $n \geq \lambda + 1$, because $\mathcal{S} \subsetneq \mathcal{A}$, and because every symbol has positive probability at every position, we have $P(E) > 0$.

On E , the sequence $\tilde{X}$ contains the maximal λ -run $(1, 1 + \lambda)$: both symbols belong to $\mathcal{S}$, the gap is λ , the run cannot be extended to the left, and it cannot be extended to the right because all other positions are non-key. Moreover,

$$\kappa(\tilde{X}_1) = \kappa(s_{\min}) = 1 \leq \kappa(\tilde{X}_{1+\lambda}).$$

Hence this maximal run is ordered, and $\tilde{X}$ is compliant.

On the same event E , the sequence $\tilde{X}'$ is not compliant. Indeed, among positions $2, \dots, n$, the only key position is $1 + \lambda$, so no compliant chain of length at least 2 can exist. Therefore

$$E \subseteq C \setminus C'.$$

Since $C' \subseteq C$, we have

$$P(C) - P(C') = P(C \setminus C').$$

Moreover, $E \subseteq C \setminus C'$ and $P(E) > 0$. Therefore

$$P(C) - P(C') \geq P(E) > 0,$$

which implies

$$P(C) > P(C').$$

Hence

$$\begin{aligned} P(Y^* = 1 \mid X_1 = s_{\min}) &= P(C) > P(C') \\ &= P(Y^* = 1 \mid X_1 = a_0). \end{aligned}$$

Case 2: $\mathcal{S} = \mathcal{A}$. When $\mathcal{S} = \mathcal{A}$, the maximal λ -runs are deterministic: they are the residue classes of positions modulo λ , truncated to $\{1, \dots, n\}$. Let

$$L := 1 + \left\lfloor \frac{n-1}{\lambda} \right\rfloor.$$

Since $n \geq \lambda + 1$, we have $L \geq 2$. Let

$$R_1 := (1, 1 + \lambda, \dots, 1 + (L-1)\lambda)$$

be the maximal λ -run containing position 1.

Let O_1 be the event that R_1 is ordered, and let B be the event that some maximal λ -run different from R_1 is ordered. Then

$$\{Y^* = 1\} = B \cup O_1.$$

Moreover, B depends only on coordinates outside R_1 , whereas O_1 depends only on coordinates in R_1 . By independence of the coordinates, for every $a \in \mathcal{A}$,

$$\begin{aligned} P(Y^* = 1 \mid X_1 = a) \\ = P(B) + (1 - P(B)) P(O_1 \mid X_1 = a). \end{aligned}$$

Since $B \subseteq \{Y^* = 1\}$, if $P(B) = 1$ then $P(Y^* = 1) = 1$, contradicting $\rho \in (0, 1)$. Hence

$$P(B) < 1.$$

Let $s_{\max} \in \mathcal{A}$ satisfy

$$\kappa(s_{\max}) = |\mathcal{A}|.$$

The assumption $\rho \in (0, 1)$ implies $|\mathcal{A}| \geq 2$, so $s_{\min} \neq s_{\max}$. Define

$$p_a := P(O_1 \mid X_1 = a).$$

We now show that $p_{s_{\min}} > p_{s_{\max}}$.

Conditioning on $X_1 = s_{\min}$, the first element of R_1 has minimal rank. Thus

$$\begin{aligned} p_{s_{\min}} &= P(\kappa(X_{1+\lambda}) \leq \kappa(X_{1+2\lambda}) \\ &\leq \dots \leq \kappa(X_{1+(L-1)\lambda})), \end{aligned}$$

with the convention that the event is the whole space when $L = 2$. Conditioning on $X_1 = s_{\max}$, the first element of R_1 has maximal rank, so the run can be ordered only if every later element of the run is also $s_{\max}$:

$$p_{s_{\max}} = P(X_{1+\lambda} = X_{1+2\lambda} = \dots = X_{1+(L-1)\lambda} = s_{\max})$$

The latter event is contained in the former. The containment is strict with positive probability, because by full support and independence the event

$$X_{1+\lambda} = X_{1+2\lambda} = \dots = X_{1+(L-1)\lambda} = s_{\min}$$

has positive probability, belongs to the former event, and does not belong to the latter. Therefore

$$p_{s_{\min}} > p_{s_{\max}}.$$

Consequently,

$$\begin{aligned} P(Y^* = 1 \mid X_1 = s_{\min}) - P(Y^* = 1 \mid X_1 = s_{\max}) \\ = (1 - P(B))(p_{s_{\min}} - p_{s_{\max}}) > 0. \end{aligned}$$

Thus

$$P(Y^* = 1 \mid X_1 = s_{\min}) \neq P(Y^* = 1 \mid X_1 = s_{\max}).$$

In both cases, there exist $a, b \in \mathcal{A}$ such that

$$P(Y^* = 1 \mid X_1 = a) \neq P(Y^* = 1 \mid X_1 = b).$$

Therefore Y^* is not independent of X_1 . Since mutual information between discrete variables is zero if and only if the variables are independent, it follows that

$$I(Y^*; X_1) > 0.$$

Finally, by the symmetric noise model (1),

$$\begin{aligned} P(Y = 1 \mid X_1 = a) \\ = P(Y = 1 \mid Y^* = 1) P(Y^* = 1 \mid X_1 = a) \\ + P(Y = 1 \mid Y^* = 0) P(Y^* = 0 \mid X_1 = a) \\ = (1 - \pi) P(Y^* = 1 \mid X_1 = a) \\ + \pi (1 - P(Y^* = 1 \mid X_1 = a)) \\ = \pi + (1 - 2\pi) P(Y^* = 1 \mid X_1 = a). \end{aligned}$$

Because $\pi < 1/2$, the map

$$u \mapsto \pi + (1 - 2\pi)u$$

is strictly increasing. Hence the map

$$a \mapsto P(Y = 1 \mid X_1 = a)$$

is also non-constant on $\mathcal{A}$. Thus Y is not independent of X_1 , and again

$$I(Y; X_1) > 0.$$

□

E.2 A Quantitative Uniform Lower Bound

Theorem 3.3 is qualitative and holds under independent full-support sampling. Under the uniform proposal law used in our synthetic benchmark, the witness event in its proof yields an explicit lower bound on the amount of local leakage.

Corollary E.2 (Uniform witness lower bound). *Assume the setting of Theorem 3.3 with $\mathcal{S} \subsetneq \mathcal{A}$ and $X_t \stackrel{\text{iid}}{\sim} \text{Uniform}(\mathcal{A})$. Let $s_{\min} \in \mathcal{S}$ satisfy $\kappa(s_{\min}) = 1$ and $a_0 \in \mathcal{A} \setminus \mathcal{S}$. Define $\delta(a, b) := P(Y^* = 1 \mid X_1 = a) - P(Y^* = 1 \mid X_1 = b)$. Then*

$$\delta(s_{\min}, a_0) \geq \frac{m}{\ell} \left(\frac{\ell - m}{\ell} \right)^{n-2}. \quad (41)$$

If Y is generated from Y^* via (1), then

$$\delta_Y(s_{\min}, a_0) \geq (1 - 2\pi) \frac{m}{\ell} \left(\frac{\ell - m}{\ell} \right)^{n-2}, \quad (42)$$

where $\delta_Y(a, b) := P(Y = 1 \mid X_1 = a) - P(Y = 1 \mid X_1 = b)$.

See E.3 for full proof and remarks.

E.3 Proof of Corollary E.2 with remarks

Proof. As in the proof of Theorem 3.3, let $Z_2, \dots, Z_n$ be independent random variables with $Z_t \sim \text{Uniform}(\mathcal{A})$, and define

$$\tilde{X} = (s_{\min}, Z_2, \dots, Z_n), \quad \tilde{X}' = (a_0, Z_2, \dots, Z_n).$$

Then

$$\tilde{X} \stackrel{d}{=} (X \mid X_1 = s_{\min}), \quad \tilde{X}' \stackrel{d}{=} (X \mid X_1 = a_0).$$

Let

$$C := \{\tilde{X} \text{ is compliant}\}, \quad C' := \{\tilde{X}' \text{ is compliant}\}.$$

As in Theorem 3.3, one has $C' \subseteq C$.

Now define the witness event

$$E := \{Z_{1+\lambda} \in \mathcal{S}\} \cap \bigcap_{t \in \{2, \dots, n\} \setminus \{1+\lambda\}} \{Z_t \notin \mathcal{S}\}.$$

On E , the sequence $\tilde{X}$ is compliant via the chain $(1, 1 + \lambda)$, while $\tilde{X}'$ is not compliant. Hence

$$E \subseteq C \setminus C'.$$

Therefore

$$P(C) - P(C') = P(C \setminus C') \geq P(E).$$

By independence and uniformity,

$$P(Z_{1+\lambda} \in \mathcal{S}) = \frac{m}{\ell}, \quad P(Z_t \notin \mathcal{S}) = \frac{\ell - m}{\ell}$$

for each $t \in \{2, \dots, n\} \setminus \{1 + \lambda\}$.

There are exactly $n - 2$ such latter positions, so

$$P(E) = \frac{m}{\ell} \left(\frac{\ell - m}{\ell} \right)^{n-2}.$$

Since

$$P(C) = P(Y^* = 1 \mid X_1 = s_{\min}), \\ P(C') = P(Y^* = 1 \mid X_1 = a_0),$$

the first claim follows.

For the observed label, by (1),

$$P(Y = 1 \mid X_1 = a) = \pi + (1 - 2\pi) P(Y^* = 1 \mid X_1 = a)$$

Subtracting the expressions for $a = s_{\min}$ and $a = a_0$ yields

$$P(Y = 1 \mid X_1 = s_{\min}) - P(Y = 1 \mid X_1 = a_0) = \\ = (1 - 2\pi) \left(P(Y^* = 1 \mid X_1 = s_{\min}) - P(Y^* = 1 \mid X_1 = a_0) \right),$$

and the result follows from the first part. □

Remark E.3 (Scope of the explicit bound). The closed-form expression in Corollary E.2 is specific to the uniform proposal law. Under the more general assumptions of Theorem 3.3, if we write

$$\alpha_t := P(X_t \in \mathcal{S}),$$

then the same witness-event argument gives

$$P(Y^* = 1 \mid X_1 = s_{\min}) - P(Y^* = 1 \mid X_1 = a_0) \\ \geq \alpha_{1+\lambda} \prod_{t \in \{2, \dots, n\} \setminus \{1+\lambda\}} (1 - \alpha_t)$$

whenever $\mathcal{S} \subsetneq \mathcal{A}$. In the i.i.d. but non-uniform case with common law q , this becomes

$$q(\mathcal{S})(1 - q(\mathcal{S}))^{n-2}.$$

No nontrivial positive lower bound depending only on (m, ℓ, n) can hold under mere full support, since $q(\mathcal{S})$ can be arbitrarily small while remaining positive.

Remark E.4 (Effect of label balancing). Suppose the final experimental sample is obtained from a proposal law P by retaining examples with probability depending only on the observed label Y . Equivalently,

$$Q(x, y) = \frac{r_y P(x, y)}{Z}, \quad r_0, r_1 > 0.$$

Then

$$Q(X | Y = y) = P(X | Y = y),$$

so class-conditional sequence structure, and hence AUC, are unchanged by this reweighting. However, posterior quantities are transformed. Writing $\beta := r_1/r_0$ and

$$h_\beta(u) := \frac{\beta u}{1 + (\beta - 1)u},$$

one has

$$Q(Y = 1 | X_1 = a) = h_\beta(P(Y = 1 | X_1 = a)).$$

Since h_β is strictly increasing, label balancing preserves the presence of local information, but because h_β is nonlinear it can either enlarge or shrink posterior gaps. Consequently, Corollary E.2 should be interpreted as a statement about the proposal law, not automatically about the retained balanced sample.

More generally, if

$$\Delta_P := P(Y = 1 | X_1 = s_{\min}) - P(Y = 1 | X_1 = a_0),$$

$$\Delta_Q := Q(Y = 1 | X_1 = s_{\min}) - Q(Y = 1 | X_1 = a_0),$$

Then by the mean value theorem:

$$\Delta_Q \geq \min\{\beta, \beta^{-1}\} \Delta_P.$$

The same remark applies with Y^* in place of Y if balancing is performed using the latent label rather than the observed one.

E.4 Proof of Theorem 3.5

Proof. For each $t \in \{1, \dots, n\}$, define

$$B_t := \mathbf{1}\{X_t \in K\}.$$

Since $X_t \stackrel{\text{iid}}{\sim} \text{Uniform}(\mathcal{A})$ and

$$p = \frac{|K|}{|\mathcal{A}|},$$

we have

$$B_t \stackrel{\text{iid}}{\sim} \text{Bernoulli}(p).$$

Let

$$T := \left(\sum_{t=1}^n B_t \right) \bmod 2.$$

We first show that Y^* is a fixed bit-flip of T . Using addition modulo 2,

$$\mathbf{1}\{N_k \text{ is even}\} = 1 \oplus (N_k \bmod 2),$$

where $\oplus$ denotes XOR. Hence

$$Y^* = 1 \oplus \bigoplus_{k \in K} \mathbf{1}\{N_k \text{ is even}\}$$

$$= 1 \oplus \bigoplus_{k \in K} (1 \oplus (N_k \bmod 2))$$

$$= 1 \oplus (|K| \bmod 2) \oplus \bigoplus_{k \in K} (N_k \bmod 2).$$

But

$$\begin{aligned} \bigoplus_{k \in K} (N_k \bmod 2) &= \left(\sum_{k \in K} N_k \right) \bmod 2 \\ &= \left(\sum_{t=1}^n \mathbf{1}\{X_t \in K\} \right) \bmod 2 = T. \end{aligned}$$

Therefore

$$Y^* = 1 \oplus (|K| \bmod 2) \oplus T.$$

The map $T \mapsto Y^*$ is thus a bijection on $\{0, 1\}$, so for every index set J ,

$$I(Y^*; X_J) = I(T; X_J).$$

It is therefore enough to study $I(T; X_J)$.

Fix an index set

$$J \subseteq \{1, \dots, n\}, \quad 1 \leq |J| < n.$$

Write

$$r := |J|, \quad s := n - r = |J^c| \geq 1.$$

Define

$$U_J := \left(\sum_{j \in J} B_j \right) \bmod 2, \quad V_J := \left(\sum_{j \in J^c} B_j \right) \bmod 2.$$

Then

$$T = U_J \oplus V_J.$$

Since $\{B_j : j \in J\}$ and $\{B_j : j \in J^c\}$ are independent, V_J is independent of X_J . Moreover, U_J is a deterministic function of X_J .

Let

$$q_s := P(V_J = 1).$$

Because V_J is the parity of s i.i.d. Bernoulli(p) random variables,

$$q_s = \frac{1 - (1 - 2p)^s}{2}.$$

Therefore, for every realization x_J of X_J ,

$$P(T = 1 | X_J = x_J) = \begin{cases} q_s, & U_J(x_J) = 0, \\ 1 - q_s, & U_J(x_J) = 1. \end{cases}$$

Balanced case: $p = 1/2$. If $p = 1/2$, then

$$q_s = \frac{1 - (1 - 2p)^s}{2} = \frac{1}{2}.$$

Hence, for every realization x_J ,

$$\begin{aligned} P(T = 1 \mid X_J = x_J) &= \frac{1}{2}, \\ P(T = 0 \mid X_J = x_J) &= \frac{1}{2}. \end{aligned}$$

Thus the conditional law of T does not depend on x_J , so T is independent of X_J . Therefore

$$I(T; X_J) = 0,$$

and hence

$$I(Y^*; X_J) = 0.$$

Unbalanced case: $p \neq 1/2$, finite n . Since K is nonempty and proper, $p \in (0, 1)$. If $p \neq 1/2$, then

$$|1 - 2p| < 1 \quad \text{and} \quad (1 - 2p)^s \neq 0$$

for every finite $s \geq 1$. Hence

$$q_s \neq \frac{1}{2}.$$

Choose any $j_0 \in J$, any $a_{\text{in}} \in K$, and any $a_{\text{out}} \in \mathcal{A} \setminus K$. Define two realizations $x_J, x'_J \in \mathcal{A}^J$ by

$$x_j = a_{\text{out}} \quad \text{for all } j \in J,$$

and

$$x'_{j_0} = a_{\text{in}}, \quad x'_j = a_{\text{out}} \quad \text{for all } j \in J \setminus \{j_0\}.$$

Then

$$U_J(x_J) = 0, \quad U_J(x'_J) = 1.$$

Therefore

$$\begin{aligned} P(T = 1 \mid X_J = x_J) &= q_s, \\ P(T = 1 \mid X_J = x'_J) &= 1 - q_s. \end{aligned}$$

Since $q_s \neq 1/2$, these two conditional probabilities are different. Thus the conditional law of T depends on X_J , so T and X_J are not independent. Hence

$$I(T; X_J) > 0,$$

and therefore

$$I(Y^*; X_J) > 0.$$

Asymptotic decay when $p \neq 1/2$. Now let $J \subset \mathbb{N}$ be any fixed finite index set, and for all sufficiently large n identify it with a subset of $\{1, \dots, n\}$. Then $r = |J|$ is fixed and

$$s = n - r \rightarrow \infty.$$

Since $|1 - 2p| < 1$,

$$q_s = \frac{1 - (1 - 2p)^s}{2} \rightarrow \frac{1}{2}.$$

Hence, uniformly over all realizations x_J ,

$$P(T = 1 \mid X_J = x_J) \rightarrow \frac{1}{2}.$$

Because X_J takes values in the finite set $\mathcal{A}^r$, this implies that the joint law of (T, X_J) converges to the product law of T and X_J . Therefore

$$I(T; X_J) \rightarrow 0.$$

Since Y^* is a bijective function of T ,

$$I(Y^*; X_J) = I(T; X_J) \rightarrow 0.$$

Observed stochastic label Y . Assume now that Y is generated from Y^* according to (1) with $\pi < 1/2$.

If $p = 1/2$, then Y^* is independent of X_J . Since Y depends on X_J only through Y^* , it follows that Y is also independent of X_J . Hence

$$I(Y; X_J) = 0.$$

If $p \neq 1/2$, then

$$\begin{aligned} P(Y = 1 \mid X_J) &= P(Y = 1 \mid Y^* = 1) P(Y^* = 1 \mid X_J) \\ &\quad + P(Y = 1 \mid Y^* = 0) P(Y^* = 0 \mid X_J) \\ &= (1 - \pi) P(Y^* = 1 \mid X_J) \\ &\quad + \pi (1 - P(Y^* = 1 \mid X_J)) \\ &= \pi + (1 - 2\pi) P(Y^* = 1 \mid X_J). \end{aligned}$$

Because $\pi < 1/2$, the map

$$u \mapsto \pi + (1 - 2\pi)u$$

is strictly increasing. Thus whenever $P(Y^* = 1 \mid X_J)$ is non-constant as a function of X_J , so is $P(Y = 1 \mid X_J)$. Therefore, for every finite n ,

$$I(Y; X_J) > 0.$$

Finally, since Y is obtained from Y^* through a channel independent of X_J , the data processing inequality gives

$$I(Y; X_J) \leq I(Y^*; X_J).$$

Hence, for every fixed finite $J \subset \mathbb{N}$,

$$I(Y; X_J) \rightarrow 0 \quad \text{as } n \rightarrow \infty.$$

This proves all claims. In particular, when $p = 1/2$, we have

$$\begin{aligned} I(Y^*; X_J) &= 0 \\ \text{for every } J \subseteq \{1, \dots, n\} \text{ with } |J| < n. \end{aligned}$$

Therefore the task is k -locally pure for every $k < n$. $\square$

F Models and Hyperparameter Configurations

This appendix provides complete hyperparameter configurations for all models evaluated in our experiments.

F.1 Training Procedure

Optimization. All neural models are trained using the AdamW optimizer [39] with weight decay. For pretrained LLMs and BERT, we use a linear learning rate schedule with 6% warmup steps. We employ cross-entropy loss for binary classification. Training uses early stopping based on validation loss with patience of 3 epochs for pretrained models and 5 epochs for models trained with randomly initialized parameters.

Pretrained models. LLMs (Llama, Qwen) and BERT are fine-tuned with learning rate 2×10^{-5} , batch size 16, and maximum sequence length 200 tokens. To enable efficient fine-tuning, we apply LoRA [40] with rank $r = 8$, $\alpha = 16$, and dropout 0.1, targeting the attention projection matrices (q-proj, k-proj, v-proj, o-proj for LLMs; query, key, value for BERT). Large models employ gradient checkpointing and mixed-precision training (bfloat16) to fit within GPU memory constraints. We train for a maximum of 20 epochs with patience of 3 epochs. Table F.2 summarizes the fine-tuning configuration for pretrained LLMs. Table F.3 shows the architectural configurations for small-scale Llama models trained with randomly initialized parameters. Table F.4 summarizes the fine-tuning configuration for BERT-based models.

Randomly initialized Models. LSTM, BiLSTM, Transformer, and hybrid architectures are trained with batch size 64 for up to 30 epochs. Learning rates are tuned per architecture: 10^{-3} for recurrent models (LSTM, BiLSTM), 5×10^{-4} for Transformer-based models, and 10^{-4} for the RNNTransformer. Dropout rates range from 0.2 to 0.3 depending on model capacity. Early stopping monitors validation F1 score with patience of 5 epochs. Table F.5 summarizes the training configuration shared across all models trained from scratch. Table F.6 and Table F.7 present the default and optimized architecture configurations, respectively.

Additional architecture details:

- **Transformer:** 4 attention heads; GELU activation; learnable positional encodings; mean pooling with padding mask.
- **LSTM:** Final hidden state used for classification.

Table F.1. Models tested and relative dimension. For XGBoost, we report hyperparameter configuration in Table F.8. As tree ensemble complexity is not directly comparable to neural network parameter counts, we omit this metric; the trained model typically comprised 80–120 trees with maximum depth 6.

Category	Model	Parameters
Pretrained	BERT-base	110M
	RoBERTa	355M
	Llama-3.2-1B	1.0B
	Qwen3-4B-Think	4.0B
	Llama-3.1-8B	8.0B
	Qwen2.5-14B	14B
Randomly initialized	Llama-3.1-70B	70B
	Llama-1M	1M
	LSTM	198K–340K
	BiLSTM	256K–395K
	Transformer	103K–104K
	RNNTransformer*	319K–455K
Tree-based	XGBoost	–

*We call RNNTransformer a BiLSTM-Transformer.

- **RNNTransformer:** BiLSTM output projected to embedding dimension, then processed by Transformer encoder.

XGBoost baselines. XGBoost models use learning rate 0.05, maximum depth 6, subsample ratio 0.8, and column sampling ratio 0.8 per tree. We apply L1 regularization ($\alpha = 0.5$) and L2 regularization ($\lambda = 1.5$) with minimum child weight 2 and minimum split gain $\gamma = 0.1$. Class imbalance is addressed via `scale_pos_weight`. Training runs for up to 200 boosting rounds with early stopping (patience 10) based on validation log-loss. Table F.8 presents the XGBoost configuration. For XGBoost-LLM, embeddings are extracted from Llama-3.2-1B using the prompt format described in Section F.2, with mean pooling over the last hidden layer (dimension 2048).

Threshold optimization. For all models, we optimize the classification threshold on the validation set by searching over $\{0.10, 0.11, \dots, 0.89\}$ to maximize F1 score, rather than using the default 0.5 threshold.

F.2 Sequence Encoding

We use three encoding strategies depending on the model family. **Prompt-based (LLMs and BERT).** For pretrained language models, we convert each sequence into a natural language prompt of the form **Sequential events:** $x_1 x_2 \dots x_n$.

For causal LLMs (Llama, Qwen), we additionally append the literal string **Outcome** (0 or 1): so that the prompt always terminates at a fixed sentinel position; the true label is never appended

Table F.2. Fine-tuning configuration for pretrained LLMs (Llama, Qwen families).

Hyperparameter	Value
<i>Training</i>	
Learning rate	2×10^{-5}
Optimizer	AdamW
Batch size	16
Max epochs	20
Early stopping patience	3
Early stopping criterion	Validation loss
Warmup ratio	6% of total steps
Scheduler	Linear decay
<i>LoRA Configuration (backbone only)</i>	
Rank (r)	8
Alpha (α)	16
Dropout	0.1
Target modules	q.proj, k.proj, v.proj, o.proj
Bias	None
PEFT task type	CAUSAL.LM (adapter-config only; see Loss row)
<i>Classification head (outside LoRA, trained full-rank)</i>	
Architecture	$d_{\text{model}} \rightarrow d_{\text{model}}/2 \rightarrow 2$
Activation / dropout	Tanh / 0.1
Read position	Last non-padding token's last-layer hidden state
<i>Loss</i>	
Total loss	$\mathcal{L}_{\text{LM}}(\text{prompt}) + \mathcal{L}_{\text{CE}}(\hat{y}_{\text{head}}, y)$
LM-loss target	The prompt itself (autoregressive on letters + template)
Inference logits	Classification head softmax (not next-token logits)
<i>Input Processing</i>	
Max sequence length	200 tokens
Padding	Max length
Truncation	True
<i>Memory Optimization</i>	
Precision	bfloat16
Gradient checkpointing	Enabled
Use cache	Disabled

to the prompt. Fine-tuning combines (i) the standard auto-regressive language-modeling loss on the prompt itself (no label leakage, equivalent to continued pretraining on the letter-sequence template) with (ii) a binary classification loss from a 2-layer MLP head ($d_{\text{model}} \rightarrow d_{\text{model}}/2 \rightarrow 2$, with Tanh and dropout 0.1) that reads from the last non-padding hidden state. The two losses are summed with equal weight; LoRA adapters cover only the backbone (q/k/v/o-proj, $r=8$), while the classification head is trained full-rank from scratch. At inference and for early-stopping, predictions come from the head's softmax, not from next-token logits. The prompt is then tokenized using each model's native tokenizer. This same prompt format is used to extract embeddings from Llama-3.2-1B for the XGBoost-LLM baseline, where we apply mean pooling over the last hidden layer.

Learned and ordinal embeddings. For all non-pretrained models, we map each letter to an integer index ($A \mapsto 1, B \mapsto 2, \dots, Z \mapsto 26$). For XGBoost-basic, this ordinal encoding is used directly as features, relying on the tree ensemble to learn sequential dependencies. For neural architectures (LSTM, Transformer, and hybrids), a learned embedding layer maps each index to a dense vector $\mathbf{e}_i \in \mathbb{R}^{d_e}$ with $d_e \in \{32, 64, 128\}$. Transformer-based models additionally incorporate learnable positional encodings $\mathbf{h}_i = \mathbf{e}_i + \mathbf{p}_i$, where $\mathbf{P} \in \mathbb{R}^{L_{\text{max}} \times d_e}$ is initialized from a standard normal distribution. Sequences are zero-padded within each mini-batch.

Table F.3. Architecture configurations for Llama models trained with randomly initialized parameters.

Model	Hidden size	Layers	Heads	KV Heads	Intermediate size
Llama-1M	128	4	4	2	512

Table F.4. Fine-tuning configuration for BERT models.

Hyperparameter	Value
<i>Training</i>	
Learning rate	2×10^{-5}
Optimizer	AdamW
Batch size	16
Max epochs	20
Early stopping patience	3
Early stopping criterion	Validation loss
Warmup ratio	6% of total steps
Scheduler	Linear decay
<i>LoRA Configuration (when used)</i>	
Rank (r)	8
Alpha (α)	16
Dropout	0.1
Target modules	query, key, value
Bias	None
Task type	SEQ_CLS
<i>Input Processing</i>	
Max sequence length	200 tokens
Padding	Max length
Truncation	True

F.3 Data Fractions

To assess data efficiency, we train each model on the following fractions of the training set: {1%, 10%, 30%, 50%, 75%, 100%}. Subsampling is performed using stratified sampling to maintain class balance across all fractions.

Reproducibility. All experiments use a fixed random seed (9550) for data splitting, weight initialization, and stochastic optimization. Each configuration is run with three different seeds; we report means and standard deviations (see Table G.1).

Table F.5. Training configuration for models trained with randomly initialized parameters.

Hyperparameter	Value
Optimizer	AdamW
Batch size	64
Max epochs	30
Early stopping patience	5
Early stopping criterion	Validation F1
Loss function	Cross-entropy

Table F.6. Default architecture configurations for randomly initialized models.

Model	Embed. dim	Hidden dim	Layers	Dropout	Learning rate
Transformer	64	256 (FFN)	2	0.3	5×10^{-4}
LSTM	32	64	2	0.3	10^{-3}
RNNTransformer*	64	64 (RNN) / 256 (FFN)	1+2	0.3	5×10^{-4}

*We call RNNTransformer a BiLSTM-Transformer.

Table F.7. Optimized architecture configurations for models randomly initialized.

Model	Embed. dim	Hidden dim	Layers	Dropout	Learning rate
Transformer	64	128 (FFN)	3	0.3	5×10^{-4}
LSTM	32	128	2	0.2	2×10^{-3}
RNNTransformer*	128	32 (RNN) / 256 (FFN)	1+3	0.3	10^{-4}

*We call RNNTransformer a BiLSTM-Transformer.

F.4 Hardware and Software

Experiments are conducted on a single NVIDIA H100 GPU (80GB). Fine-tuning Llama-8B requires approximately 128GB CPU memory with gradient checkpointing enabled. Training times range from minutes (small models, 1% data) to several hours (Llama-8B, full data).

Table F.8. XGBoost hyperparameter configuration.

Hyperparameter	Value
<i>Tree Parameters</i>	
Max depth	6
Min child weight	2
Gamma (min split loss)	0.1
Subsample	0.8
Column sample by tree	0.8
<i>Regularization</i>	
L1 regularization (α)	0.5
L2 regularization (λ)	1.5
<i>Training</i>	
Learning rate (η)	0.05
Max boosting rounds	200
Early stopping patience	10
Early stopping criterion	Validation log-loss
Tree method	hist
<i>Class Imbalance</i>	
Scale pos weight	$N_{\text{negative}}/N_{\text{positive}}$

G Results

This appendix provides the numerical results of our framework and the cohort-construction details and audit results for the real-world CKD→ESRD diagnosis-sequence experiment in Section 4.4.

G.1 Real-world audit

The goal is not to build a clinical deployment model, but to test whether temporal order is decisive for model performance under a diagnosis-code representation.

Cohort definition. We use MIMIC-IV v3.1 diagnosis records [24, 41]. ICD diagnosis codes are mapped to Clinical Classifications Software (CCS) categories [44]. CKD is defined using ICD-10 N18.1–N18.5 and ICD-9 585.1–585.5. ESRD-related end-point codes include ICD-10 N18.6, Z99.2, Z49.* and ICD-9 585.6, V45.1, V56.*. Positives are patients whose first ESRD-related code occurs in an admission strictly after the first CKD-coded admission; their sequences are truncated before the ESRD-coded admission. Negatives are CKD patients with no ESRD-related code in the observed record. Patients whose first CKD and ESRD-related codes occur in the same admission are excluded to avoid same-visit label leakage. Patients with fewer than 10 diagnosis codes after preprocessing are also excluded. Across admissions, diagnoses are ordered by admission time; within admissions, they are ordered by diagnosis sequence number, which reflects billing/discharge order rather than true event time. See Table G.2 for an overview.

Table G.1. Full classification performance across models and datasets (100% training data). Standard deviations in parentheses.

Model	AUC	F1	Prec	Rec
<i>Naive ($\pi = 0.0$)</i>				
Oracle	1.000	1.000	1.000	1.000
k-gram	.999	.999	.999	.999
XGBoost (basic)	.603 (.004)	.714 (.005)	.556 (.004)	.995 (.003)
XGBoost (LLM)	.502 (.004)	.536 (.008)	.501 (.004)	.577 (.010)
BERT	.999 (.001)	.999 (.001)	.999 (.001)	.999 (.001)
RoBERTa	.999 (.001)	.999 (.001)	.999 (.001)	.999 (.001)
Llama-3.2-1B	.999 (.001)	.999 (.001)	.999 (.001)	.999 (.001)
Llama-3.1-8B	.999 (.001)	.999 (.001)	.999 (.001)	.999 (.001)
Llama-1M (cold)	.999 (.001)	.999 (.001)	.999 (.001)	.999 (.001)
Qwen3-4B-Think	.999 (.001)	.999 (.001)	.999 (.001)	.999 (.001)
LSTM	.999 (.001)	.999 (.001)	.999 (.001)	.999 (.001)
Transformer	.999 (.001)	.999 (.001)	.999 (.001)	.999 (.001)
RNNTransformer [†]	.999 (.001)	.999 (.001)	.999 (.001)	.999 (.001)
<i>OC-Deterministic ($\pi = 0.0$)</i>				
Oracle	1.000	1.000	1.000	1.000
2-gram	.821	.672	.593	.775
XGBoost (basic)	.625 (.012)	.457 (.015)	.516 (.010)	.409 (.018)
XGBoost (LLM)	.494 (.010)	.399 (.012)	.353 (.008)	.459 (.015)
BERT	.999 (.001)	.999 (.001)	.999 (.001)	1.000 (.001)
RoBERTa	.999 (.001)	.999 (.001)	.999 (.001)	1.000 (.001)
Llama-3.2-1B	.846 (.015)	.695 (.018)	.623 (.020)	.785 (.022)
Llama-3.1-8B	.995 (.003)	.984 (.008)	.976 (.010)	.993 (.007)
Llama-1M (cold)	.832 (.012)	.686 (.015)	.605 (.018)	.791 (.020)
Qwen3-4B-Think	.850 (.014)	.701 (.016)	.620 (.017)	.800 (.020)
LSTM	.999 (.001)	.994 (.003)	.995 (.003)	.994 (.003)
Transformer	.999 (.001)	.992 (.002)	.985 (.005)	1.000 (.001)
RNNTransformer [†]	.997 (.002)	.992 (.003)	.985 (.005)	1.000 (.002)
<i>OC-Noisy ($\pi = 0.3$)</i>				
Oracle	.670	.577	.700	.491
2-gram	.585	.592	.420	1.000
XGBoost (basic)	.463 (.010)	.418 (.012)	.437 (.008)	.400 (.015)
XGBoost (LLM)	.418 (.011)	.453 (.014)	.437 (.008)	.470 (.018)
BERT	.669 (.006)	.592 (.004)	.420 (.005)	1.000 (.002)
RoBERTa	.652 (.008)	.592 (.003)	.420 (.005)	1.000 (.002)
Llama-3.2-1B	.587 (.008)	.592 (.005)	.420 (.006)	.999 (.005)
Llama-3.1-8B	.575 (.008)	.591 (.005)	.425 (.006)	.968 (.010)
Qwen3-4B-Think	.597 (.007)	.591 (.004)	.423 (.005)	.979 (.008)
Qwen3-2.5-14B*	.670 (—)	.586 (—)	.476 (—)	.763 (—)
LSTM	.665 (.003)	.562 (.020)	.660 (.002)	.489 (.031)
Transformer	.671 (.001)	.579 (.003)	.701 (.015)	.492 (.012)
RNNTransformer [†]	.671 (.005)	.578 (.005)	.683 (.010)	.501 (.005)
<i>GPC-Deterministic ($\pi = 0.0$)</i>				
Oracle	1.000	1.000	1.000	1.000
1-gram	.505	.668	.501	1.000
XGBoost (basic)	.495 (.004)	.490 (.005)	.497 (.003)	.484 (.006)
XGBoost (LLM)	.501 (.003)	.639 (.005)	.502 (.002)	.881 (.008)
BERT	.503 (.002)	.668 (.002)	.501 (.002)	1.000 (.001)
RoBERTa	.501 (.003)	.668 (.002)	.501 (.002)	1.000 (.001)
Llama-3.2-1B	.497 (.005)	.668 (.001)	.501 (.002)	1.000 (.001)
Llama-3.1-8B	.500 (.003)	.668 (.002)	.501 (.002)	1.000 (.001)
Llama-1M (cold)	.503 (.003)	.668 (.002)	.501 (.003)	1.000 (.001)
Llama-3.1-70B*	.502 (—)	.668 (—)	.501 (—)	1.000 (—)
Qwen3-4B-Think	.500 (.002)	.668 (.001)	.501 (.000)	1.000 (.001)
LSTM	.502 (.003)	.668 (.002)	.501 (.002)	1.000 (.001)
Transformer	.501 (.003)	.668 (.002)	.501 (.002)	1.000 (.001)
RNNTransformer [†]	.501 (.002)	.668 (.002)	.501 (.003)	1.000 (.001)

*single due to time constraint, no standard deviation.

[†]We call RNNTransformer a BiLSTM-Transformer.

auc_combined_full_1row.png not found in this checkout.

Full AUC grid across all 13 models on the three synthetic datasets.

Figure G.1. Full AUC learning curves for all evaluated models on the three core tasks. Left: OC-Deterministic ($\pi = 0$); middle: OC-Noisy with symmetric label noise ($\pi = 0.3$); right: hidden-subset GPC-Deterministic ($\pi = 0$). AUC is used because it is threshold-independent and is the primary metric for rule recovery; threshold-dependent metrics are reported separately in Table G.1. Dashed horizontal lines indicate the relevant reference levels: oracle AUC for deterministic compliance, the noise-limited oracle AUC for OC-Noisy, and chance performance for GPC-Deterministic. The curves show that encoder-style and recurrent models reach the compliance ceiling under our protocol, some decoder-only models plateau below that ceiling on the OC datasets, and all tested architectures remain at chance on hidden-subset GPC-Deterministic under single-stage end-to-end supervision. Shaded regions denote variability across seeds; single-run large-model results are plotted without shading.

Table G.2. Cohort summary and sanity checks for the MIMIC-IV CKD→ESRD audit.

Category	Quantity	Value
Cohort	Patients	14,813
	Progressors, $Y = 1$	1,146
	Non-progressors, $Y = 0$	13,667
	Progressor prevalence	7.7%
	CCS vocabulary size	293
Sequence length	Median / mean / max codes	49 / 82 / 2,396
	Median length, $Y = 1$ / $Y = 0$	54 / 49
Admissions	Median / mean admissions	3 / 4.9
	Patients with ≥ 2 admissions	76%
	Patients with ≥ 3 admissions	58%
Overlap checks	Shared CCS categories	265 / 293
	Vocabulary Jaccard overlap	0.90
	CCS categories exclusive to $Y = 1$	0
	Cosine similarity of code frequencies	0.956
	Spearman correlation of code frequencies	0.968

CCS_158, is highly prevalent in both groups and therefore only weakly enriched. In contrast, the more enriched unigrams are shared diagnosis categories that occur more frequently among progressors. This supports the interpretation that the task is not driven by progressor-exclusive codes, but by frequency differences among shared clinical content.

Interpretation. The audit shows that CKD→ESRD prediction, while is clinically temporal, is not strongly order-informational under this diagnosis-code representation. Unigrams and bag-of-codes baselines are already strong, and sequential-model gains largely survive order destruction. This should not be interpreted as evidence that CKD progression is clinically non-temporal, nor

Audit protocol. We apply three shortcut controls. First, for each contiguous k -gram g , we estimate $\hat{P}(Y = 1 | g)$ on the training set and score each test patient by averaging over the patient’s observed k -grams. Second, we train order-invariant bag-of-codes Logistic Regression and XGBoost models over CCS counts. Third, for sequence models, we compare ordered training/evaluation against a shuffled condition that randomly permutes each patient’s diagnosis sequence while preserving the patient-level multiset of codes. See Table G.3 for the results.

Discriminative unigram content. In the MIMIC-IV audit, each token X_t is a CCS diagnosis category. A k -gram is therefore a contiguous block of k CCS tokens, and the $k = 1$ case corresponds to a single diagnosis category. To understand why unigram and bag-of-codes baselines are strong, we inspect patient-level CCS prevalence differences between progressors and non-progressors. Table G.4 reports selected enriched unigrams. These are descriptive content checks, not validated clinical trajectories.

The cohort-defining CKD category itself,

Table G.3. Compact audit summary for MIMIC-IV CKD→ESRD. k -gram rows report mean $\pm$ standard deviation over five 80/20 stratified splits. Bag-of-codes and neural rows use the 60/20/20 neural-model split.

Audit	Model/control	Ordered AUC	Shuffled AUC	Auxiliary	Interpretation
k -gram	1-gram	0.823 ± 0.015	–	F1 0.356	Code identity/frequency carries strong signal.
	2-gram	0.801 ± 0.016	–	F1 0.341	Adding contiguous order does not improve over unigrams.
	3-gram	0.576 ± 0.015	–	F1 0.165	Longer local patterns become sparse.
	4–7 gram	$0.479\text{--}0.500$	–	F1 0.144–0.146	Higher-order k -grams collapse to chance.
Bag-of-codes	Logistic Reg.	0.809	0.809	$\Delta = 0$	Strong order-invariant baseline.
	XGBoost	0.803	0.803	$\Delta = 0$	Strong order-invariant baseline.
Sequence models	Transformer	0.860	0.844	$\Delta = +0.016$	Most of the gain survives order destruction.
	BiLSTM	0.820	0.823	$\Delta = -0.003$	No positive ordered-vs-shuffled gain.
	BERT-base	0.826	0.843	$\Delta = -0.017$	No positive ordered-vs-shuffled gain.
	LSTM	0.500	0.467	failed	Did not converge reliably on long sequences.

Table G.4. Selected CCS unigrams enriched among CKD→ESRD progressors. Frequencies are patient-level prevalences. The log ratio is $\log_2(\text{freq}_{Y=1}/\text{freq}_{Y=0})$.

Code	Description	$Y = 1$	$Y = 0$	$\log_2$ ratio
CCS.156	Nephritis, nephrosis, renal sclerosis	30.3%	9.2%	+1.71
CCS.87	Retinal detachments, retinopathy	18.1%	7.4%	+1.29
CCS.215	Genitourinary congenital anomalies	5.5%	2.0%	+1.47
CCS.158	Chronic kidney disease	99.7%	89.9%	+0.15

H Sensitivity analysis (full sweep)

We run the stochastic sweep over $m \in \{3, 6, 10\}$ and $\lambda \in \{1, 3, 5, 7, 9, 10\}$, yielding 18 configurations. For each configuration we train BERT-base, a single-layer LSTM, a two-layer Transformer encoder, and Llama-3.2-1B with LoRA from scratch on 400K training sequences, using the same hyperparameters as Appendix F.1; we report mean AUC over three seeds and divide by the theoretical oracle AUC^* from Theorem E.1 to obtain the AUC/AUC^* ratio used in Table H.1.

BERT and LSTM stay within 0.05 of the ceiling across the sweep.

Three patterns emerge from Table H.1. First, BERT and LSTM are essentially at the oracle ceiling throughout the stochastic sweep. Second, Llama-3.2-1B is consistently below the encoder/recurrent models, with the largest gaps in the $m = 10$ regime where AUC^* is highest. Third, the small Transformer is sensitive to dense-key regimes: it degrades at $m = 10$ for small and medium lags but recovers at larger lags. The main-paper setting ($m = 6, \lambda = 7$) is therefore a medium-difficulty point rather than a cherry-picked worst case; harder $m = 10$ regimes amplify the encoder/recurrent advantage.

as evidence that sequence models are inadequate. Rather, it indicates that this representation contains strong non-temporal shortcuts: diagnosis identity, frequency, and co-occurrence carry enough signal that temporal order is not decisive for prediction.

Limitations and data access. MIMIC-IV is an ICU-centered EHR dataset and may not reflect outpatient CKD progression trajectories. The audit uses diagnosis codes only; richer longitudinal variables such as eGFR, laboratory values, medications, dialysis timing, visit intervals, and outpatient follow-up may contain decisive temporal information. Within-admission order reflects billing/discharge order rather than clinical event time. All MIMIC-IV analyses are performed under credentialed access. We release cohort-construction and preprocessing code, but not raw MIMIC-IV data or patient-level derived sequences.

Table H.1. Sensitivity analysis in the stochastic setting ($\pi = 0.3$). Each cell reports AUC/AUC*, where AUC* is the per-configuration theoretical ceiling. A ratio of 1.00 indicates the model reaches the Bayes-optimal bound.

m	λ	AUC*	BERT	LSTM	Transformer	Llama-3.2-1B
3	1	.602	1.00	1.00	0.94	0.96
3	3	.595	1.00	1.00	0.94	0.93
3	5	.589	1.00	1.00	0.94	0.95
3	7	.581	1.00	0.98	0.96	0.95
3	9	.574	1.00	1.00	0.97	0.96
3	10	.570	1.00	0.99	1.00	0.95
6	1	.683	1.00	1.00	0.97	0.88
6	3	.679	1.00	1.00	0.99	0.86
6	5	.675	1.00	1.00	1.00	0.86
6	7	.670	1.00	0.99	1.00	0.88
6	9	.666	1.00	0.99	1.00	0.88
6	10	.664	1.00	0.99	1.00	0.89
10	1	.699	1.00	1.00	0.76	0.78
10	3	.699	1.00	1.00	0.80	0.79
10	5	.698	1.00	0.99	0.83	0.81
10	7	.698	1.00	0.96	0.87	0.85
10	9	.697	1.00	0.96	0.98	0.85
10	10	.696	1.00	0.98	0.99	0.87

Bold: baseline configuration ($m=6, \lambda=7$) from the main paper.

I Mechanism-identification audit

This appendix documents the mechanism-identification audit referenced in Section 5.1 (baseline ladder, λ -agnostic ladder, held-out-rule generalization, parity decomposition) together with two complementary analyses: a binary-parity reference protocol locating our regime relative to known learnable cases, and an in-context few-shot evaluation.

I.1 Baseline ladder on standard evaluation

We report test AUC across seven feature families and two classifiers (logistic regression with L_2 regularization, XGBoost) on the released OC-Deterministic and OC-Noisy splits. Training sample size: 200 000; evaluation: 50 000. Families include full 26-dimensional letter counts, 6-dimensional key-letter counts, residue-class counts modulo λ , aggregated λ -aware pair counts ($g_{ab}^{(\lambda=7)}$), 6×6 key-only lag-pair counts, position-aware lag-pair indicators (sparse, 8788-dim), and $6 \times 6 \times 6$ lag-trigram counts.

I.2 λ -agnostic ladder

To test whether the encoder’s advantage comes from knowing λ , we stack pair counts across all offsets $\lambda' \in \{1, \dots, n-1\}$, giving a 12 844-dimensional feature. L_1 logistic regression on this feature space reaches AUC = 0.997 on OC-Deterministic (recovering $\lambda = 7$ from data; the per- λ weight norm is 303 at $\lambda = 7$ versus ≤ 48 elsewhere) and AUC = 0.632

on OC-Noisy (where label noise prevents clean lag identification, with the top-weight lag drifting to $\lambda = 19$). XGBoost on the same features closes the gap on OC-Noisy, reaching AUC = 0.667, essentially matching the best encoder.

I.3 Held-out-rule generalization

To check whether the λ -aware pair feature carries a transferable algorithm or a per-rule linear weighting, we generate 6 synthetic datasets with random $(\mathcal{S}, \kappa, \lambda)$ drawn from the same family as the main OC-Noisy dataset. Leave-one-rule-out: train logistic regression on 5 rules, evaluate on the held-out rule and on a held-out split of the same rule. The λ -aware pair-count feature reaches AUC = 1.00 within-rule and AUC = 0.50 across-rule; full letter-count features reach AUC = 0.77 within-rule and AUC = 0.49 across-rule. The linear ceiling is therefore carried by a per-rule weighting, not by a transferable procedure.

I.4 GPC-Deterministic failure decomposition

For the GPC-Deterministic case, we decompose its failure by revealing successive levels of information about the hidden subset K . Given raw tokens with K hidden, both logistic regression and a 64-unit MLP remain at chance (AUC = 0.50). Revealing the per-position membership bit $b_t = \mathbf{1}\{X_t \in K\}$ (a 20-dimensional binary input) allows the same MLP to reach AUC = 0.997 in under one minute of training while logistic regression remains at chance. Revealing the 6-dimensional per-key count vector or per-key parity bits reaches AUC = 0.997 and 1.000 respectively with the MLP. Revealing only the scalar total key count yields AUC = 0.981 with the MLP.

I.5 Reproduction of a binary-parity reference protocol

As defined in Definition 3.4, the GPC mechanism is based on counting and checking whether the count is even. A known setting in which small transformer encoders learn parity is binary-vocabulary parity on short sequences [25]. To locate our main-paper regime relative to this boundary, we reproduce that setting exactly, a 2-layer Transformer encoder ($d_{\text{model}} = 64$, 8 heads, vocabulary size 2, $n = 10$) trained for 1000 steps with a [CLS] readout, and then sweep sequence length and data regime to characterize the transition.

Experimental grid. We vary (i) sequence length $n \in \{10, 20\}$ and (ii) training-data regime: *streaming* (a fresh uniformly random batch of 1024 sequences at every gradient step) or *fixed* (a training pool of

Table I.1. Test AUC for binary parity ($\ell = 2$, $|K| = 1$) across sequence length and training-data regime. Values are mean (std) over 10 seeds, following the format of Table G.1. Success rate counts seeds reaching AUC > 0.9 . At $n = 10$, the same three seeds ($\{2, 4, 8\}$) fail in both regimes, confirming that the outcome is controlled by model initialization rather than the data-sampling regime.

Regime	n	AUC	Success rate
streaming	10	.850 (.242)	7/10
fixed	10	.850 (.241)	7/10
streaming	20	.503 (.004)	0/10
fixed	20	.499 (.005)	0/10

100,000 sequences sampled once and drawn from with replacement at every step). All runs use identical architecture, optimizer (Adam, $\text{lr} = 10^{-3}$), batch size (1024), step count (1000), and are repeated for seeds $\in \{0, 1, \dots, 9\}$. Test AUC is measured on a held-out set of 10,000 fresh sequences. Because outcomes at $n = 10$ are bimodal (a run either converges to AUC ≈ 1.0 or remains at chance), we report both the mean $\pm$ std and the success rate, defined as the fraction of seeds reaching AUC > 0.9 .

Three observations from Table I.1. (i) Streaming versus fixed-size training makes no material difference at either sequence length. A fixed pool of 10^5 binary sequences of length 10 contains the full $2^{10} = 1024$ -element input space many times over, and even at $n = 20$ the fixed-size and streaming curves coincide at chance; data-regime effects are therefore not the load-bearing variable. (ii) At $n = 10$, 7 of 10 seeds reach AUC ≈ 1.0 and 3 remain at chance, and the identity of the failing seeds ($\{2, 4, 8\}$) is identical across the two regimes. Since our seed controls model initialization and only the data pipeline differs across regimes, this is strong evidence that the failure mode is initialization-driven, consistent with the known sensitivity of parity learning to initialization in small transformers. (iii) At $n = 20$, all 20 runs across both regimes remain at AUC ≈ 0.50 , matching the main-paper result in the vocabulary sweep at $\ell = 2$, $|K| = 1$.

Takeaway. The protocol demonstrates that the balanced ablation at $n = 20$, $|K| = \ell/2$ lies on the failing side of a length-driven phase boundary, while the $n = 10$ reference point lies on the learnable side. This suggests that sequence length and class balance are important contributors, not a methodological artifact of our training pipeline, consistent with length-dependent parity difficulty in standard-depth self-attention architectures [25].

Table I.2. Test AUC on parity at the balanced regime $|K| = \max(1, \lfloor \ell/2 \rfloor)$, $n = 20$. Mean (std) over 3 seeds.

Arch	$\ell=2$	$\ell=4$	$\ell=8$	$\ell=16$	$\ell=26$
BERT	.508 (.008)	.497 (.001)	.501 (.004)	.499 (.003)	.498 (.008)
LSTM	.501 (.003)	.499 (.002)	.499 (.003)	.503 (.001)	.505 (.002)
Transformer	.505 (.000)	.501 (.003)	.500 (.002)	.504 (.003)	.501 (.005)

I.6 Vocabulary-size sweep at the balanced regime

Setup. We sweep the alphabet size $\ell \in \{2, 4, 8, 16, 26\}$ at the balanced parity regime $|K| = \max(1, \lfloor \ell/2 \rfloor)$, so that $|K|/|\mathcal{A}| = 1/2$ at every ℓ and Theorem 3.5 gives exact k -local purity for every $k < n$. Sequence length is fixed at $n = 20$. For each ℓ we generate 100K train, 20K validation, and 20K test sequences with i.i.d. uniform sampling over the ℓ -letter alphabet (data builder: `src/data/parity_variants.py --build_anchor`).

Models and training. Three architectures: BERT-base (110M, fine-tuned with LoRA, 20 epochs, batch 32); a 2-layer Transformer encoder ($d_{\text{model}} = 64$, 8 heads, ~ 170 K parameters); and a single-layer LSTM. The two from-scratch architectures are trained for 30 epochs with batch 64, AdamW, learning rate 10^{-3} . Three seeds per cell.

Takeaway. Every architecture sits at chance for every alphabet size in the balanced regime, including the binary case $\ell = 2$, $|K| = 1$. This is consistent with Theorem 3.5: at $|K|/|\mathcal{A}| = 1/2$ the task is k -locally pure for every $k < n$, so none of our tested architectures trained end-to-end without curriculum at $n = 20$ finds a useful learning signal. The length-driven phase boundary documented for binary parity in Appendix I.5 (success at $n = 10$, failure at $n = 20$) is therefore the relevant explanation for the main-paper failure, not a vocabulary effect. Results are reported in Table I.2.

I.7 Subset-size sweep at $\ell = 26$

Setup. We fix $\ell = 26$ and $n = 20$ and sweep $|K| \in \{2, 4, 6, 8, 10, 13\}$, so that $p = |K|/|\mathcal{A}|$ ranges from $2/26 \approx 0.077$ to $13/26 = 0.500$ (the exactly balanced point). Per cell: 100K train, 20K validation, 20K test. Same architectures and training schedule as Appendix I.6, three seeds per cell.

Takeaway. The transition is sharp and located at $|K| = 2$, not at the balanced point. At $|K| = 2$ all three architectures reach AUC ≈ 1.000 ($p = 2/26 \approx 0.077$ provides large finite- n local leakage, which bounded-attention models pick up in full); from $|K| = 4$ onward every architecture drops to chance and stays there through $|K| = 13$. The main-paper

Table I.3. Test AUC on parity at $\ell = 26$, $n = 20$ as $|K|$ varies. Mean (std) over 3 seeds. $|K| = 13$ is the exactly balanced point $p = 1/2$ at which Theorem 3.5 gives exact k -local purity.

Arch	$ K =2$	$ K =4$	$ K =6$	$ K =8$	$ K =10$	$ K =13$
BERT	1.000 (.000)	.497 (.003)	.501 (.008)	.499 (.003)	.505 (.002)	.498 (.008)
LSTM	1.000 (.000)	.501 (.001)	.505 (.005)	.504 (.003)	.502 (.001)	.505 (.002)
Transformer	1.000 (.000)	.504 (.001)	.501 (.001)	.500 (.002)	.505 (.003)	.501 (.005)

Table I.4. Test AUC on parity at the balanced regime $\ell = 26$, $|K| = 13$ as n varies. Mean (std) over 3 seeds.

Arch	$n=10$	$n=15$	$n=20$	$n=30$
BERT	.501 (.001)	.501 (.007)	.498 (.008)	.503 (.004)
LSTM	.498 (.004)	.499 (.001)	.505 (.002)	.503 (.003)
Transformer	.500 (.002)	.498 (.001)	.501 (.005)	.497 (.004)

regime ($|K| = 6$, $p \approx 0.231$) sits on the failing side of this transition, and approaching the balanced point does not change the outcome. The result is consistent with the bounded-local statement of Theorem 3.5: for $p \neq 1/2$ finite- n leakage is positive but vanishes for fixed local view as n grows; here $n = 20$ is already large enough that any $|K| \geq 4$ regime is no longer learnable end-to-end at our training budget. Results are reported in Table I.3.

I.8 Sequence-length sweep at the balanced regime

Setup. We fix the balanced regime $\ell = 26$, $|K| = 13$ ($p = 1/2$, exact k -local purity by Theorem 3.5) and sweep $n \in \{10, 15, 20, 30\}$. The $n = 20$ column reuses the anchor runs from the $|K| = 13$ cell of Appendix I.7; for $n \in \{10, 15, 30\}$ we generate fresh 100K train / 20K validation / 20K test splits at $\ell = 26$, $|K| = 13$. Same architectures and training schedule as Appendix I.7, three seeds per cell.

Takeaway. At the balanced large-alphabet regime every architecture sits at chance across all four lengths, including $n = 10$. This is the key contrast with the binary regime ($\ell = 2$, $|K| = 1$) of Appendix I.5, where the same 2-layer Transformer reaches AUC ≈ 1.0 for 7/10 seeds at $n = 10$ and decays to chance only at $n = 20$. The balanced-and-large-alphabet task appears harder under our tested training protocol: at $\ell = 26$, $|K| = 13$ even the short-sequence regime is not learned end-to-end at our training budget. Theorem 3.5 removes the local shortcuts captured by our bounded-view analysis: reducing n from 20 to 10 is not enough to compensate when the task is exactly k -locally pure. Results are reported in Table I.4.

I.9 In-context (few-shot) evaluation

To rule out that the model-performance patterns of Table G.1 are artefacts of the fine-tuning access

Table I.5. In-context (few-shot) evaluation on OC-Deterministic, OC-Noisy, and GPC-Deterministic. Values are AUC mean (std) over 3 shot-selection seeds; -- marks 0-shot runs, which are deterministic given the test-set subsample (single seed). Format matches Table G.1; compare the cells below to the corresponding fine-tuned rows there.

Task	Model	0-shot	5-shot	20-shot
<i>OC-Deterministic</i>				
	Llama-3.2-1B	.468 (—)	.516 (.010)	.544 (.028)
	Llama-3.1-8B	.507 (—)	.540 (.034)	.564 (.075)
	Qwen2.5-14B	.451 (—)	.511 (.014)	.531 (.052)
<i>OC-Noisy</i>				
	Llama-3.2-1B	.512 (—)	.496 (.016)	.502 (.014)
	Llama-3.1-8B	.514 (—)	.525 (.023)	.517 (.013)
	Qwen2.5-14B	.486 (—)	.492 (.009)	.509 (.013)
<i>GPC-Deterministic</i>				
	Llama-3.2-1B	.533 (—)	.505 (.010)	.497 (.014)
	Llama-3.1-8B	.503 (—)	.496 (.008)	.502 (.010)
	Qwen2.5-14B	.480 (—)	.503 (.002)	.506 (.009)

mode, we re-evaluate the three largest decoder-only models from our study in an in-context-learning configuration without any parameter updates, and compare to their fine-tuned counterparts on matched test splits.

Protocol. For each model $M \in \{\text{Llama-3.2-1B, Llama-3.1-8B, Qwen2.5-14B}\}$, task $\tau \in \{\text{OC-Deterministic, OC-Noisy, GPC-Deterministic}\}$, and shot count $k \in \{0, 5, 20\}$, we sample k class-balanced training sequences (seeds $\{0, 1, 2\}$ for $k > 0$), render the same raw-variant prompt used for our CausalLM fine-tuning pipeline (“**Sequential events:** <letters>\nOutcome (0 or 1):”), and perform a single forward pass per test sequence. We read the logits for the single-token IDs of “_0” and “_1” at the final position, softmax between the two, and use the resulting $P(Y=1)$ to compute AUC, F_1 , and accuracy. Test set: 1000 examples per (τ, M, k) cell, drawn once from the paper test split per seed; this gives AUC within ± 0.015 at 95% CI, similar to the precision of the fine-tuned rows of Table G.1.

Three observations from Table I.5. (i) *On OC-Deterministic*, in-context performance rises modestly with k (average lift of +0.08 AUC from 0- to 20-shot) but no model reaches AUC = 0.6, whereas fine-tuned Llama-3.1-8B reaches AUC = 0.995 and fine-tuned BERT reaches 0.999 on the same test split. Scale within the ICL regime (1B $\rightarrow$ 14B) moves the 20-shot number only from 0.544 to 0.531, whereas switching from ICL to fine-tuning moves it from 0.564 to 0.995; the dominant axis of variation is access mode, not model size. (ii) *On OC-Noisy*, whose oracle AUC* = 0.670 is saturated by fine-tuned BERT, no ICL configuration exceeds

AUC = 0.525; all three models remain essentially at chance regardless of k . ICL fails to extract even the stochastic-regime signal that fine-tuning extracts in full. (iii) *On GPC-Deterministic*, every (M, k) cell is at chance, matching the fine-tuned parity column of Table G.1 where every architecture sits at AUC ≈ 0.50 .

Takeaway. The fine-tuned-to-ICL gap is large on the two *learnable* tasks (OC-Deterministic, OC-Noisy) and vanishes on the *unlearnable* one (GPC-Deterministic). This is the behavior we expect if the bottleneck is the discovery of the latent combinatorial structure (a λ -aware pair count on the OC datasets; joint hidden-subset identification and modular arithmetic on GPC-Deterministic), rather than the training regime used to extract it. The ICL access mode therefore amplifies rather than causes the performance patterns of Table G.1.

I.10 Length-based curriculum protocol and full per-seed results

Setup. We test a two-stage length curriculum on main-paper parity ($\ell = 26$, $K = \mathcal{S}_{\text{parity}}$, $|K| = 6$). Stage 1 trains a model from random initialization on $n = 10$ sequences. Stage 2 loads the matched-seed Stage 1 checkpoint and fine-tunes on $n = 20$. The shape mismatch in the positional embedding $([1, 10, d] \rightarrow [1, 20, d])$ is handled by copying the first 10 positions from the Stage 1 checkpoint and randomly initializing the remaining 10; all other parameters are copied directly. Each stage uses a freshly generated dataset of 100K train / 20K validation / 20K test sequences.

Architectures and hyperparameters.

- **Transformer:** 2 layers, 8 heads, $d_{\text{model}} = 64$, $d_{\text{ff}} = 512$, dropout 0.3 (168 642 parameters at $n = 10$, 169 282 at $n = 20$). Adam optimizer, learning rate 10^{-3} , batch size 64, 30 epochs, early-stopping patience 3 on validation loss.
- **LSTM:** same training schedule as the Transformer.
- **BERT-base:** 110M parameters, fine-tuned with LoRA following Appendix F; 20 epochs, batch size 32.

We run 3 seeds per architecture per stage.

Per-seed results. Table I.6 reports test AUC for every (architecture, stage, seed) cell. Three patterns are visible. First, the LSTM is fully insensitive to initialization on this task: all 6 runs reach AUC ≥ 0.999 . Second, the Transformer’s Stage 2 outcome is perfectly determined by its Stage 1 outcome:

Table I.6. Length-based curriculum learning per-seed test AUC. Stage 1: cold-start training on $n = 10$. Stage 2: warm-start fine-tuning on $n = 20$ from the matched-seed Stage 1 checkpoint.

Architecture	Seed	Stage 1 ($n=10$)	Stage 2 ($n=20$)
Transformer	0	.999	.999
	1	.502	.497
	2	.999	.999
LSTM	0	1.000	1.000
	1	1.000	1.000
	2	1.000	1.000
BERT	0	.507	.493
	1	.504	.502
	2	.508	.500

seed 1, which fails to learn $n = 10$ parity (AUC = 0.502), also fails after curriculum at $n = 20$ (AUC = 0.497); seeds 0 and 2, which succeed at $n = 10$, also succeed at $n = 20$ (AUC ≈ 1.0 in both stages). The init-driven bimodality documented for $n = 10$ binary parity in Appendix I.5 is therefore inherited unchanged by the curriculum. Third, BERT-base does not benefit from the protocol: all 6 BERT runs sit at AUC ≈ 0.50 , and in particular Stage 1 itself is unsolved (no seed clears 0.51 at $n = 10$). The curriculum cannot transfer a representation that the warm-start checkpoint does not contain, so the Stage 2 outcome is at chance for every seed.

J Exploratory attention inspection

We include an attention inspection as a qualitative diagnostic only; our mechanism attribution is based on the feature-space audits in Section 5.1, not on attention as causal explanation. We extract per-layer attention matrices from fine-tuned BERT-base and Llama-3.2-1B on 10,000 held-out OC-Deterministic test sequences. For each source position containing a key letter, we partition unmasked target positions into three classes: (i) lag-connected key targets at distance λ , (ii) other key targets, and (iii) non-key targets. We then average attention mass over heads and examples within each layer. Figure J.1 reports the resulting profiles on true-compliant sequences correctly predicted by each model.

Two qualitative patterns emerge. First, neither model concentrates attention on the exact lag-connected key pairs in the middle or deeper layers. In BERT, lag-connected attention is high only in the first two layers and becomes negligible by the layer where other key-to-key attention peaks; in Llama-3.2-1B, lag-connected attention is also not selectively elevated and is typically the smallest of the three classes. Thus, successful compliance classification does not appear to require direct attention mass

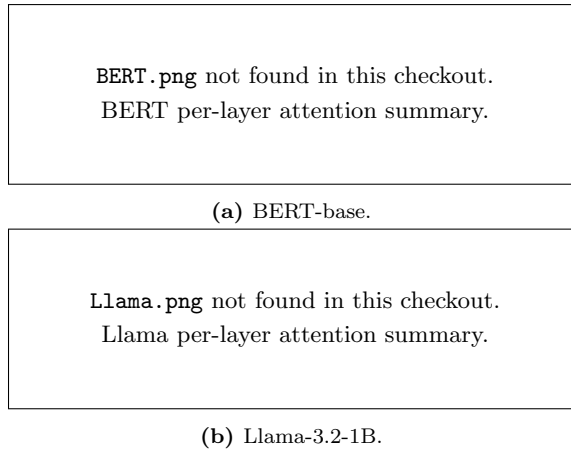


Figure J.1. Per-layer attention allocation from key source positions on true-compliant sequences correctly predicted by each model. Red: lag-connected key targets at distance λ among unmasked positions. Blue: other key targets. Gray: non-key targets. BERT shows key/non-key separation in middle layers, but neither model shows selective attention to lag-connected key pairs. Panels use separate y -axis ranges; because BERT and Llama have different attention masks and architectures, absolute magnitudes should not be compared directly.

from a key position to its lag- λ counterpart.

Second, BERT shows a visible key/non-key separation that Llama-3.2-1B does not: from about layer 4 onward, BERT assigns less attention to non-key targets than to other key targets, whereas in Llama the non-key and other-key curves remain close. This is consistent with the audit result that compliance contains exploitable key-symbol and lagged-pair signal. However, because the models have different attention masks, architectures, and readout mechanisms, we do not interpret absolute attention magnitudes as a controlled architecture comparison.

We also stratified true-compliant sequences into correctly predicted and missed examples. The same averaged statistic did not cleanly separate the two groups for either model: correct predictions did not show a systematic increase in attention to lag-connected key pairs. We therefore do not interpret residual errors as failures of attending to the right positions alone. Identifying a causal pathway would require value-pathway, MLP, and readout ablations.